Chương 10: State Pattern – Trạng thái vạn vật



State Pattern – Trạng thái của vạn vật

State Pattern là gì? Hãy ngầm hiểu đây là một mẫu đại diện cho các trạng thái khác nhau của một đối tượng. Ví dụ một thang máy sẽ có trạng thái: đóng cửa, mở cửa, chuông báo động, số tầng…hay một vận động viên bơi lội sẽ có các trạng thái: khởi động, vào vị trí, sẵn sàng, bơi, chạm đích…chúng đều có liên quan đến mẫu thiết kế được đề cập trong chương này: Mẫu Trạng thái – State Pattern.

Một thực tế ít được biết đến: Strategy và State Pattern là cặp song sinh được tách ra khi chào đời

Như bạn đã biết, Strategy Pattern (https://toihocdesignpattern.com/chuong-1-strategy-pattern-chao-mung-den-voi-design-patterns.html) đã tiếp tục tạo ra một doanh nghiệp cực kỳ thành công xung quanh các thuật toán có thể hoán đổi cho nhau. Tuy nhiên, **State Pattern** đã đi con đường có lẽ cao quý hơn là giúp

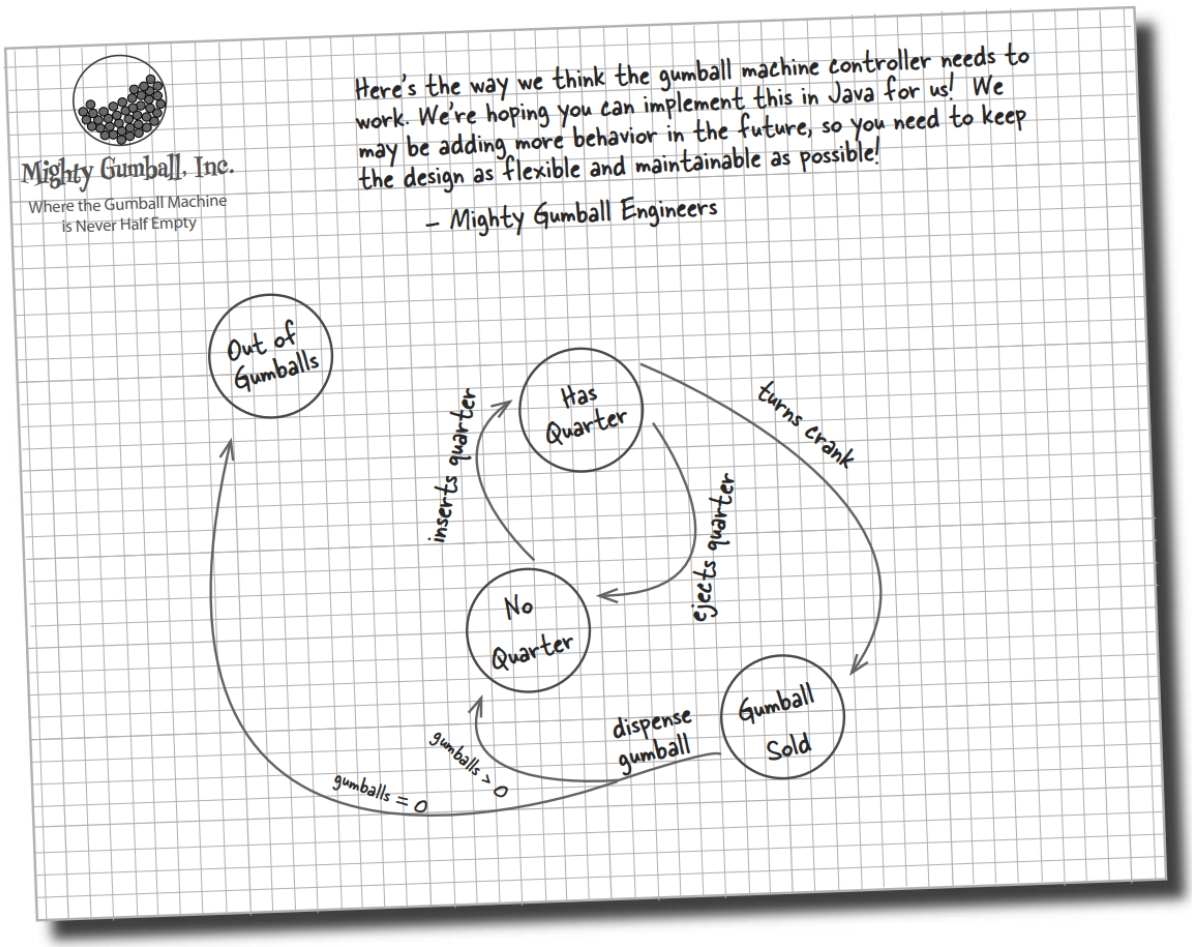
các đối tượng kiểm soát hành vi bằng cách thay đổi trạng thái nội bộ của chúng. State pattern thường lắng nghe được những object client của mình, "chỉ cần lặp lại sau tôi: Tôi đã đủ tốt, tôi đủ thông minh, và hãy chú ý…"



Ngày nay mọi người đang đưa công nghệ Java vào các thiết bị thực (IoT), như máy kẹo cao su. Đúng vậy, máy kẹo cao su đã đi vào công nghệ; các nhà sản xuất lớn đã phát hiện ra rằng bằng cách đưa CPU vào những cái máy đó, họ có thể tăng doanh số, theo dõi hàng tồn kho thông qua network và đo lường sự hài lòng của khách hàng chính xác hơn.

Ít nhất đó là câu chuyện của họ – chúng tôi nghĩ rằng họ đã cảm thấy nhàm chán với công nghệ những năm 1800 và cần tìm cách để làm cho công việc thú vị hơn.

Nhưng các nhà sản xuất này là các chuyên gia máy kẹo cao su, không phải nhà phát triển phần mềm và họ đã yêu cầu sự giúp đỡ của bạn:



(*Chú thích: Quarter là đồng 25 xu của Mỹ, crank là tay quay trên máy bắn kẹo, Gumball là kẹo cao su*)



Anne: Sơ đồ này trông giống như một sơ đồ trạng thái.

Joe: Phải, mỗi vòng tròn đó là một trạng thái…

Anne: …và mỗi mũi tên là một chuyển tiếp trạng thái.

Frank: Chậm lại, hai bạn, tôi đã nghiên cứu sơ đồ trạng thái từ quá lâu. Bạn có thể nhắc tôi chúng là gì không?

Anne: Chắc chắn rồi, Frank. Nhìn vào các vòng tròn; Đó là những trạng thái. "**No Quarter**" (Không có xu) có thể là trạng thái khởi đầu cho máy bắn kẹo cao su vì nó chỉ chờ "insert quarter" (bỏ đồng xu vào). Tất cả các vòng tròn chỉ là các trạng thái khác nhau của máy hoạt động theo một cách nhất định và cần một số hành động để đưa chúng đến trạng thái khác.

Joe: phải rồi, để đi đến một trạng thái khác, bạn cần phải làm một cái gì đó như bỏ một đồng xu vào máy. Xem mũi tên từ "**No Quarter**" đến "**Has Quarter**".

Frank: Vâng …



Đây là Gumball Machine

Joe: Điều đó có nghĩa là nếu **Gumball Machine** đang ở trạng thái "**No Quarter**" và bạn bỏ vào một đồng xu, nó sẽ đổi thành trạng thái "**Has Quarter**". Đó là sự chuyển đổi trạng thái.

Frank: Ồ, tôi hiểu rồi! Và nếu tôi đang ở trạng thái "**Has Quarter**", tôi có thể xoay tay quay và đổi sang trạng thái "**Gumball Sold**" , hoặc "**Eject quarter**" (trả lại đồng xu) sau đó chuyển về trạng thái "**No Quarter**".

Anne: Bạn đã hiểu rồi đấy!

Frank: Điều này có vẻ không tệ lắm. Chúng ta rõ ràng có bốn trạng thái, và tôi nghĩ rằng chúng ta cũng có bốn hành động: "insert quarter – chèn xu", "eject quarter – đẩy xu ra", "turn crank – xoay tay quay" và "dispense – phân phát kẹo". Nhưng khi chúng tôi trả kẹo về, trong trạng thái "Gumball Sold", chúng ta kiểm tra còn kẹo trong máy hay không, và sau đó chuyển sang trạng thái "**Out of Gumballs**" khi hết kẹo hoặc trạng thái "**No Quarter**" khi còn kẹo. Vì vậy, chúng ta thực sự có năm chuyển đổi từ trạng thái này sang trạng thái khác.

Anne: Kiểm tra máy còn kẹo cao su hay không cũng ngụ ý rằng chúng ta cũng phải theo dõi số lượng kẹo. Bất cứ khi nào máy cung cấp cho bạn một viên kẹo cao su, nó có thể là viên cuối cùng, và nếu đúng, chúng ta cần chuyển sang trạng thái "**Out of Gumballs** – hết kẹo".

Joe: Ngoài ra, đừng quên rằng bạn còn có thể làm những việc như từ chối (eject) đồng xu khi máy bắn kẹo cao su ở trạng thái "**No Quarter**", hoặc khi chèn hai đồng xu.

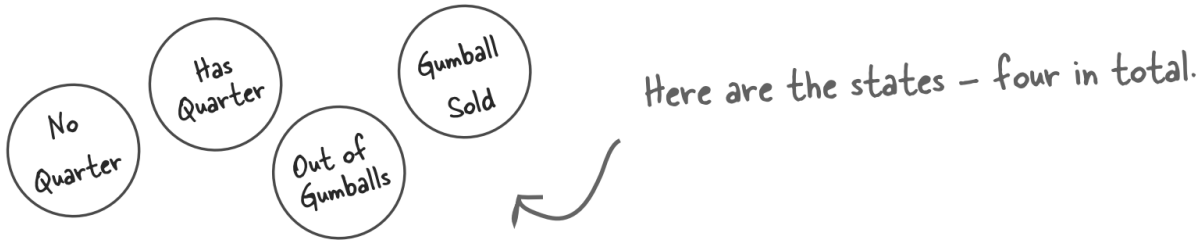
Frank: Ồ, tôi đã không nghĩ về điều này; chúng ta cũng sẽ phải quan tâm những thứ đó.

Joe: Đối với mọi hành động có thể, chúng ta sẽ phải kiểm tra xem chúng ta đang ở trạng thái nào và hành động phù hợp tiếp theo. Chúng ta có thể làm được việc này! Hãy bắt đầu ánh xạ sơ đồ trạng thái sang code…

State machines 101

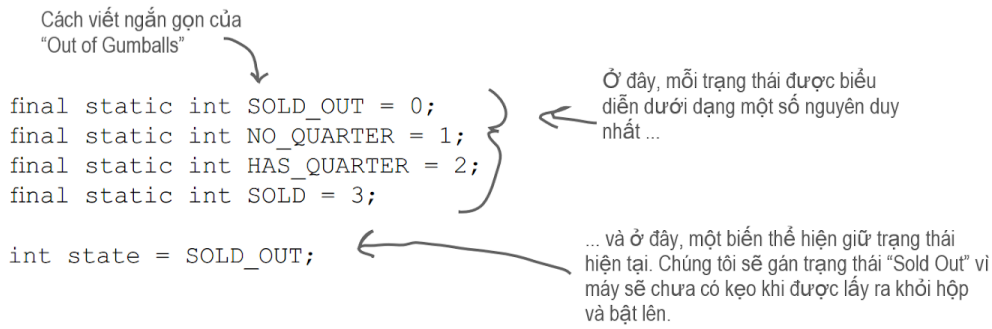
Làm thế nào chúng ta chuyển từ sơ đồ trạng thái sang code thực tế? Ở đây, giới thiệu nhanh về việc triển khai các state machines (trạng thái máy bắn kẹo):

1. Đầu tiên, tập hợp các trạng thái của bạn:

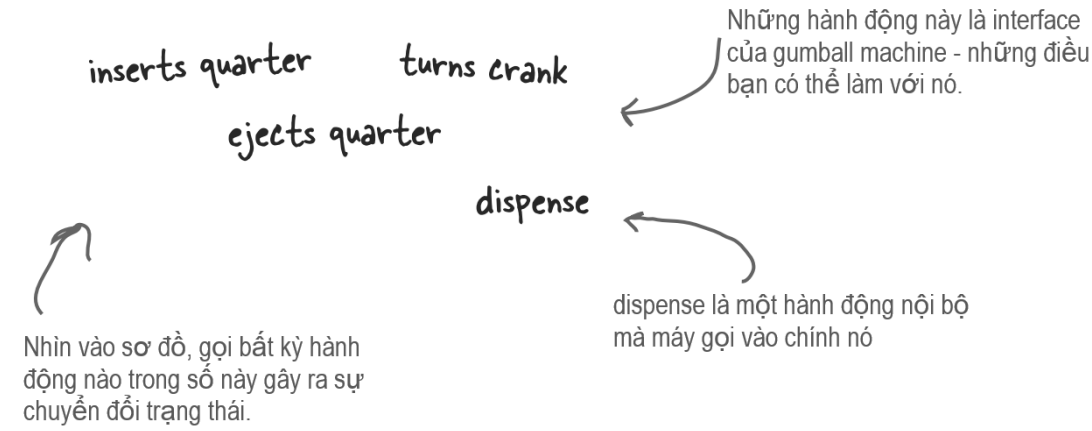


Tổng cộng có 4 trạng thái: Không có xu, Có xu, Hết kẹo, Đã phân phát kẹo.

2. Tiếp theo, tạo một biến đối tượng để giữ trạng thái hiện tại và xác định các giá trị cho từng trạng thái:



3. Bây giờ chúng tôi tập hợp tất cả các hành động có thể xảy ra trong hệ thống:



insers quater (bỏ xu vào), turns crank (xoay tay quay), ejects quarter (trả xu lại), dispense (phân phát kẹo)

4. Bây giờ chúng ta tạo một lớp hoạt động như state machine. Đối với mỗi hành động, chúng tôi tạo ra một phương thức sử dụng các câu lệnh có điều kiện để xác định hành vi nào phù hợp ở mỗi trạng thái. Ví dụ, đối với hành động chèn đồng xu, chúng ta có thể viết một phương thức như thế này:

```
public void insertQuarter() {

    if (state == HAS_QUARTER) {

        System.out.println("You can't insert another quarter");

    } else if (state == SOLD_OUT) {

        System.out.println("You can't insert a quarter, the machine is sold out");

    } else if (state == SOLD) {

        System.out.println("Please wait, we're already giving you a gumball");

    } else if (state == NO_QUARTER) {

        state = HAS_QUARTER;
        System.out.println("You inserted a quarter");

    }
}
```

Mỗi trạng thái có thể được kiểm tra với một statement có điều kiện...

... Và thể hiện hành vi phù hợp cho từng trạng thái...

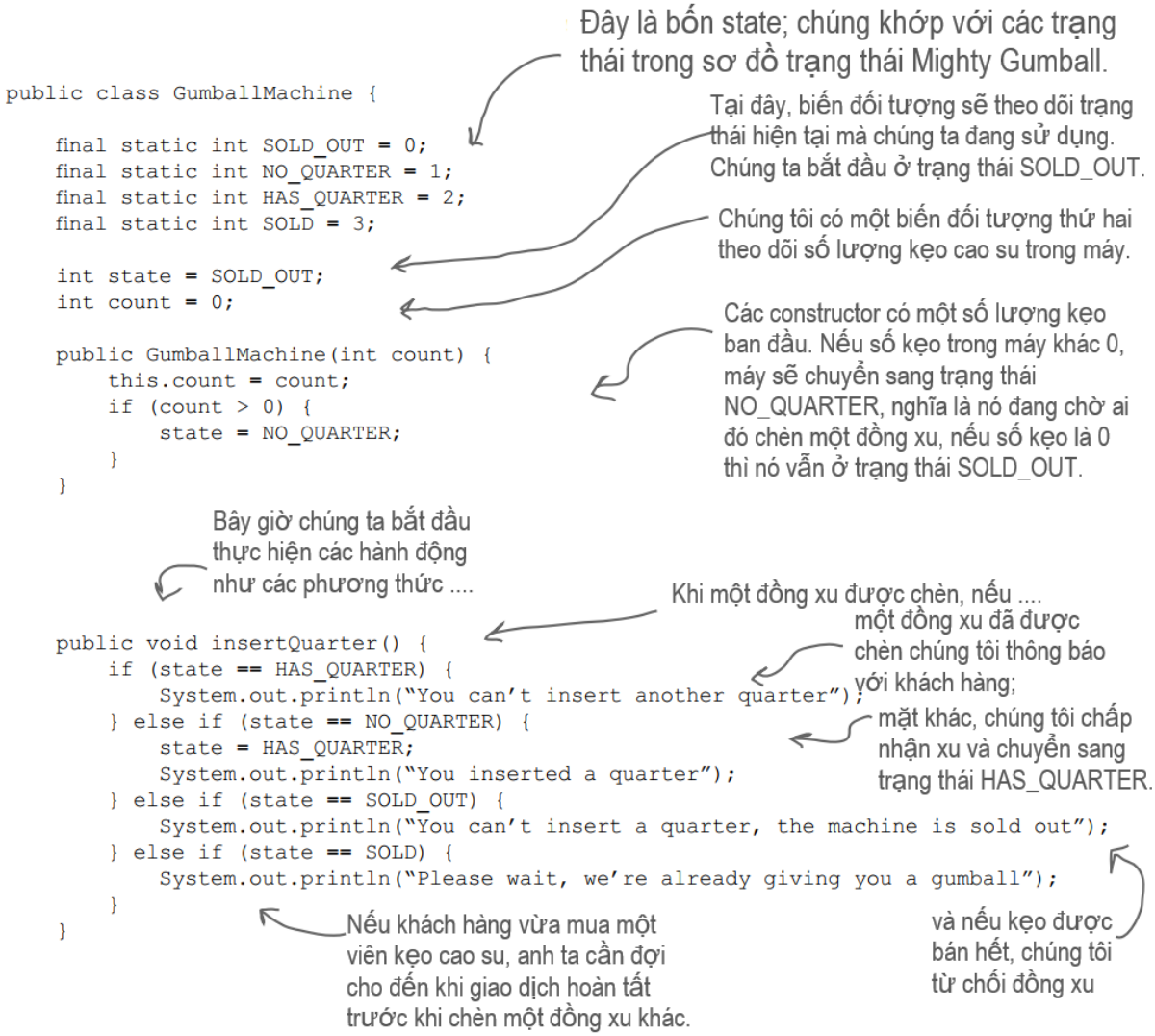
... nhưng cũng có thể chuyển sang các trạng thái khác, giống như được mô tả trong sơ đồ.



Viết code

Đây là thời gian để thực hiện **Gumball Machine**. Chúng tôi biết rằng chúng tôi sẽ có một biến đối tượng giữ trạng thái hiện tại (current state). Từ đó, chúng ta chỉ cần xử lý tất cả các hành động, hành vi và chuyển trạng thái có thể xảy ra. Đối với các hành động, chúng ta cần thực hiện chèn một đồng xu, trả lại

một đồng xu, xoay tay quay và phân phát một viên kẹo cao su; chúng tôi cũng có kiểm tra điều kiện khi Gumball machine hết kẹo.




```
public class GumballMachine {

    final static int SOLD_OUT = 0;
    final static int NO_QUARTER = 1;
    final static int HAS_QUARTER = 2;
    final static int SOLD = 3;

    int state = SOLD_OUT;
    int count = 0;

    public GumballMachine(int count) {
        this.count = count;
        if (count > 0) {
            state = NO_QUARTER;
        }
    }

    public void insertQuarter() {
        if (state == HAS_QUARTER) {
            System.out.println("You can't insert another quarter");
        } else if (state == NO_QUARTER) {
            state = HAS_QUARTER;
            System.out.println("You inserted a quarter");
        } else if (state == SOLD_OUT) {
            System.out.println("You can't insert a quarter, the machine is sold out");
        } else if (state == SOLD) {
            System.out.println("Please wait, we're already giving you a gumball");
        }
    }
```




```
public void ejectQuarter() {
    if (state == HAS_QUARTER) {
        System.out.println("Quarter returned");
        state = NO_QUARTER;
    } else if (state == NO_QUARTER) {
        System.out.println("You haven't inserted a quarter");
    } else if (state == SOLD) {
        System.out.println("Sorry, you already turned the crank");
    } else if (state == SOLD_OUT) {
        System.out.println("You can't eject, you haven't inserted a quarter yet");
    }
}

public void turnCrank() {
    if (state == SOLD) {
        System.out.println("Turning twice doesn't get you another gumball!");
    } else if (state == NO_QUARTER) {
        System.out.println("You turned but there's no quarter");
    } else if (state == SOLD_OUT) {
        System.out.println("You turned, but there are no gumballs");
    } else if (state == HAS_QUARTER) {
        System.out.println("You turned...");
        state = SOLD;
        dispense();
    }
}

public void dispense() {
    if (state == SOLD) {
        System.out.println("A gumball comes rolling out the slot");
        count = count - 1;
        if (count == 0) {
            System.out.println("Oops, out of gumballs!");
            state = SOLD_OUT;
        } else {
            state = NO_QUARTER;
        }
    } else if (state == NO_QUARTER) {
        System.out.println("You need to pay first");
    } else if (state == SOLD_OUT) {
        System.out.println("No gumball dispensed");
    } else if (state == HAS_QUARTER) {
        System.out.println("No gumball dispensed");
    }
}

// other methods here like toString() and refill()
```

In-house testing

Cảm giác đó giống như một thiết kế chắc chắn đẹp bằng cách sử dụng một phương pháp được suy nghĩ kỹ lưỡng phải không? Hãy kiểm tra một chút trước khi chúng tôi đưa nó cho công ty Mighty Gumball để được nạp vào máy bắn kẹo cao su thực tế. Ở đây, thử nghiệm của chúng tôi:

```
public class GumballMachineTestDrive {
    public static void main(String[] args) {
        GumballMachine gumballMachine = new GumballMachine(5);

        System.out.println(gumballMachine);

        gumballMachine.insertQuarter();
        gumballMachine.turnCrank();

        System.out.println(gumballMachine);

        gumballMachine.insertQuarter();
        gumballMachine.ejectQuarter();
        gumballMachine.turnCrank();

        System.out.println(gumballMachine);

        gumballMachine.insertQuarter();
        gumballMachine.turnCrank();
        gumballMachine.insertQuarter();
        gumballMachine.turnCrank();
        gumballMachine.ejectQuarter();

        System.out.println(gumballMachine);

        gumballMachine.insertQuarter();
        gumballMachine.insertQuarter();
        gumballMachine.turnCrank();
        gumballMachine.insertQuarter();
        gumballMachine.turnCrank();
        gumballMachine.insertQuarter();
        gumballMachine.turnCrank();

        System.out.println(gumballMachine);
    }
}
```

Khởi tạo nó lên với tổng số năm viên kẹo cao su

In ra trạng thái của máy

Chèn một đồng xu vào...

Xoay tay quay; chúng ta nên lấy kẹo cao su

In ra trạng thái của máy, một lần nữa.

Chèn một đồng xu vào

Yêu cầu trả lại đồng xu

Xoay tay quay; chúng ta không thể lấy kẹo

In ra trạng thái của máy, một lần nữa.

Chèn một đồng xu vào

Xoay tay quay, chúng ta sẽ nhận kẹo

Chèn một đồng xu vào

Xoay tay quay, chúng ta nên lấy kẹo

Yêu cầu trả lại đồng xu khi trước đó không chèn đồng xu vào

In ra trạng thái của máy, một lần nữa.

Chèn 1 lúc 2 đồng xu vào

Xoay tay quay, chúng ta sẽ nhận kẹo

Bây giờ thời gian để kiểm tra lại

In trạng thái máy đó thêm một lần nữa.

```
File Edit Window Help mightygumball.com
%java GumballMachineTestDrive

Mighty Gumball, Inc.
Java-enabled Standing Gumball Model #2004
Inventory: 5 gumballs
Machine is waiting for quarter

You inserted a quarter
You turned...
A gumball comes rolling out the slot

Mighty Gumball, Inc.
Java-enabled Standing Gumball Model #2004
Inventory: 4 gumballs
Machine is waiting for quarter

You inserted a quarter
Quarter returned
You turned but there's no quarter

Mighty Gumball, Inc.
Java-enabled Standing Gumball Model #2004
Inventory: 4 gumballs
Machine is waiting for quarter

You inserted a quarter
You turned...
A gumball comes rolling out the slot
You inserted a quarter
You turned...
A gumball comes rolling out the slot
You haven't inserted a quarter

Mighty Gumball, Inc.
Java-enabled Standing Gumball Model #2004
Inventory: 2 gumballs
Machine is waiting for quarter

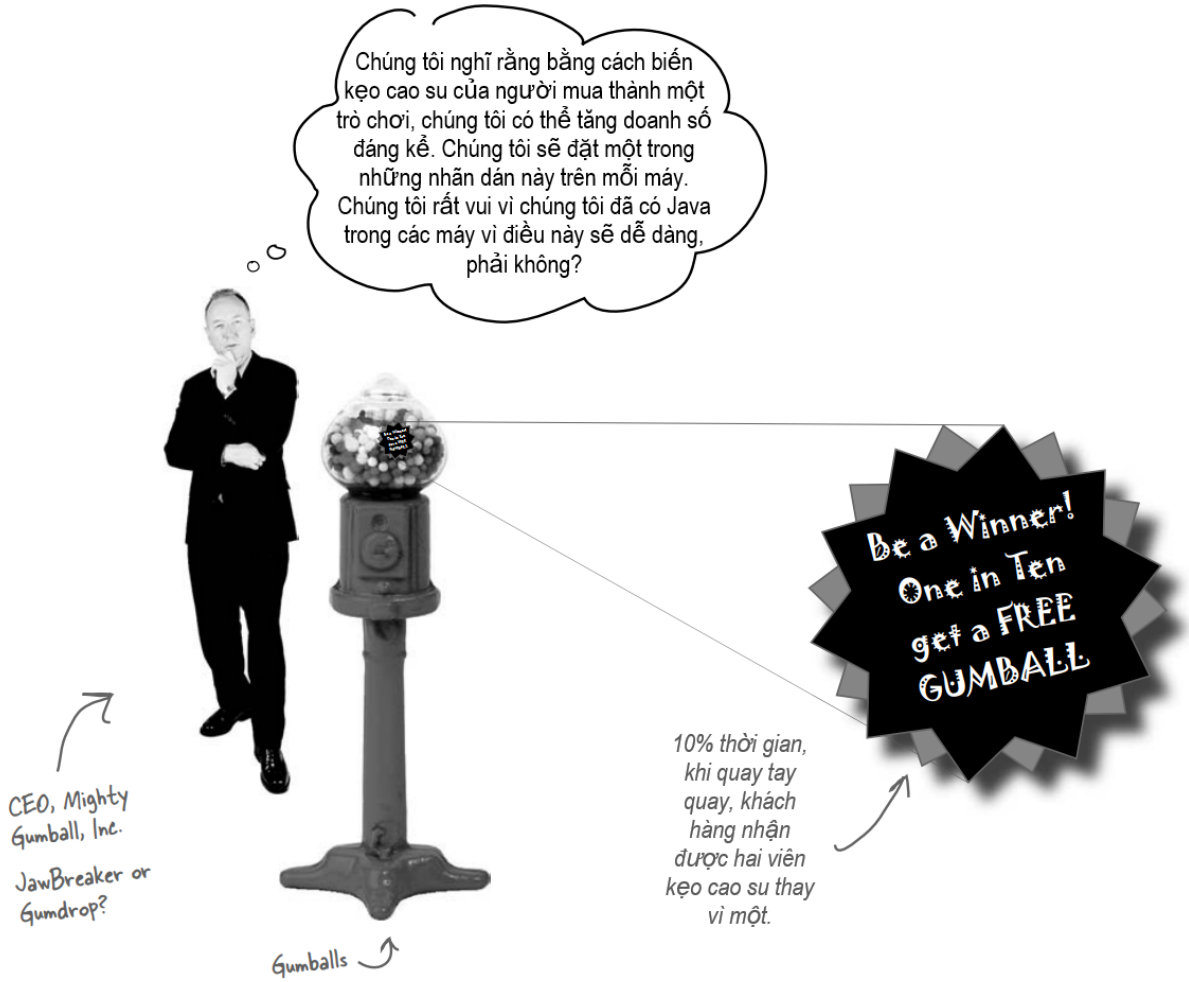
You inserted a quarter
You can't insert another quarter
You turned...
A gumball comes rolling out the slot
You inserted a quarter
You turned...
A gumball comes rolling out the slot
Oops, out of gumballs!
You can't insert a quarter, the machine is sold out
You turned, but there are no gumballs

Mighty Gumball, Inc.
Java-enabled Standing Gumball Model #2004
Inventory: 0 gumballs
Machine is sold out
```

Bạn biết rằng nó đang đến...một yêu cầu thay đổi!

Mighty Gumball, Inc. đã cài đặt code của bạn vào Gumball machine và các chuyên gia đảm bảo chất lượng của Mighty Gumball, Inc. đang chạy code thông qua các trạng thái của máy. Cho đến hiện tại, mọi thứ đều trông rất tuyệt từ góc nhìn của họ.

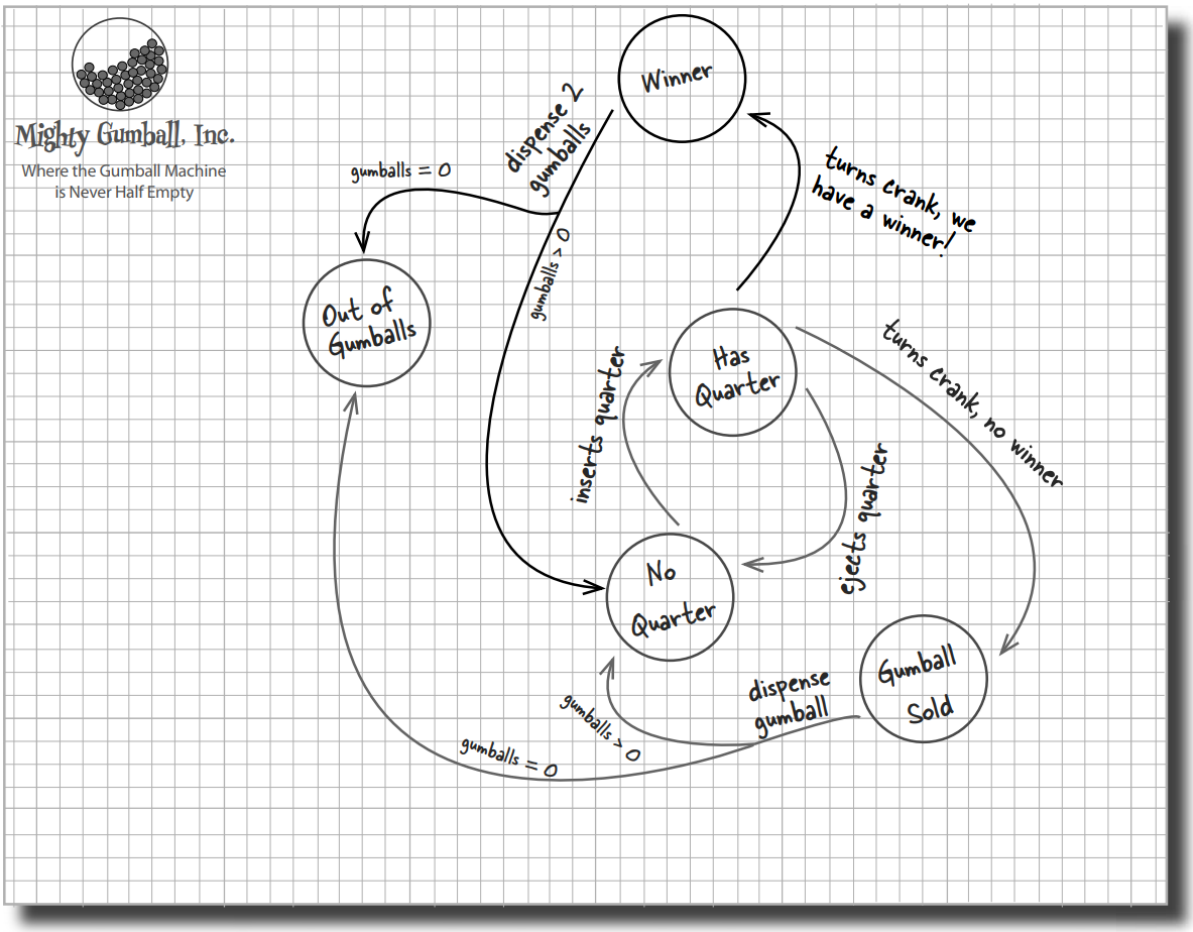
Trên thực tế, mọi thứ đã diễn ra suôn sẻ, họ muốn đưa mọi thứ lên một tầm cao mới…



Bài tập:

Vẽ sơ đồ trạng thái cho Gumball Machine xử lý trò chơi "1 phần 10". Trong trò chơi này, 10% thời gian trạng thái Sold dẫn đến hai viên kẹo được phân phát chứ không phải là một. Kiểm tra câu trả lời của bạn với chúng tôi để đảm bảo chúng tôi đồng ý trước khi bạn đi xa hơn…

Đáp án:



Các trạng thái hiện tại rất lộn xộn

Bạn đã viết code cho Gumball machine bằng phương pháp được suy nghĩ kỹ lưỡng không có nghĩa là nó sẽ dễ dàng được mở rộng. Thực tế, khi bạn quay lại và xem code của mình và suy nghĩ về những gì bạn sẽ phải làm để sửa đổi nó, thì…

```
final static int SOLD_OUT = 0;
final static int NO_QUARTER = 1;
final static int HAS_QUARTER = 2;
final static int SOLD = 3;


public void insertQuarter() {
    // insert quarter code here
}

public void ejectQuarter() {
    // eject quarter code here
}

public void turnCrank() {
    // turn crank code here
}

public void dispense() {
    // dispense code here
}
```

Đầu tiên, bạn phải thêm một trạng thái WINNER mới tại đây. Điều đó không tệ lắm...

...nhưng sau đó, bạn phải thêm một điều kiện mới vào mọi phương thức để xử lý trạng thái WINNER; mà rất nhiều code để sửa đổi.

turnCrank() sẽ trở nên đặc biệt lộn xộn, bởi vì bạn phải thêm code để kiểm tra xem liệu bạn đã có một người chiến thắng và sau đó chuyển sang trạng thái WINNER hoặc trạng thái SOLD.

Điều nào sau đây mô tả trạng thái thực hiện của chúng tôi? (Chọn tất cả các đáp án đúng.)

- ❑ A. Code này chắc chắn không tuân thủ Nguyên tắc Đóng mở.
- ❑ B. Code này sẽ làm cho một lập trình viên FORTRAN tự hào.
- ❑ C. Thiết kế này không phải là hướng đối tượng.
- ❑ C. Chuyển trạng thái không rõ ràng; họ được giấu giữa một loạt các statement có điều kiện.
- ❑ D. Ở đây, húng tôi chưa đóng gói bất cứ những gì khác biệt.
- ❑ E. Các bổ sung khác có khả năng gây ra lỗi trong code làm việc.

Đáp án: A,B,C,D,E



Joe: Bạn có thể nói đúng về điều đó! Chúng ta cần cấu trúc lại (refactor) code này để nó dễ bảo trì và sửa đổi.

Anne: Chúng ta thực sự nên cố gắng "độc lập hóa" hành vi cho từng trạng thái để nếu có một trạng thái cần thay đổi, chúng ta sẽ không gây ảnh hưởng đến các đoạn code khác.

Joe: Phải; nói cách khác, hãy làm theo điều đó.

Anne: Chính xác.

Joe: Nếu chúng ta đặt mỗi hành vi state trong lớp riêng của nó, thì mỗi state chỉ thực hiện các hành động của riêng mình.

Anne: Phải. Và có lẽ Gumball Machine có thể ủy thác hành động cho state object đại diện cho trạng thái hiện tại.

Joe: Ah, bạn nói đúng: luôn ủng hộ composition… nhiều nguyên tắc hơn trong công việc.

Anne: Dễ thương đấy. Chà, tôi không chắc chắn 100% việc này sẽ hoạt động như thế nào, nhưng tôi nghĩ chúng ta đã làm điều gì đó đúng.

Joe: Tôi tự hỏi điều này sẽ làm cho việc thêm các trạng thái mới dễ dàng hơn không?

Anne: Tôi nghĩ vậy… Chúng ta vẫn phải thay đổi code, nhưng những thay đổi sẽ bị giới hạn phạm vi hơn nhiều vì thêm trạng thái mới sẽ có nghĩa là chúng ta chỉ cần thêm một lớp mới và có thể thay đổi một vài chuyển đổi ở đó.

Joe: Tôi thích nghe điều đó. Hãy bắt đầu thiết kế mới này!

Thiết kế mới với State Pattern

Có vẻ như chúng tôi đã có một kế hoạch mới: thay vì duy trì code hiện có của chúng tôi, chúng tôi sẽ làm lại nó để đóng gói các đối tượng trạng thái trong các lớp của riêng chúng và sau đó ủy thác cho trạng thái hiện tại khi xảy ra hành động.

Chúng tôi sẽ làm theo các nguyên tắc thiết kế, vì vậy chúng tôi sẽ hoàn thành với một thiết kế dễ bảo trì hơn. Đây là cách mà chúng tôi sẽ làm:

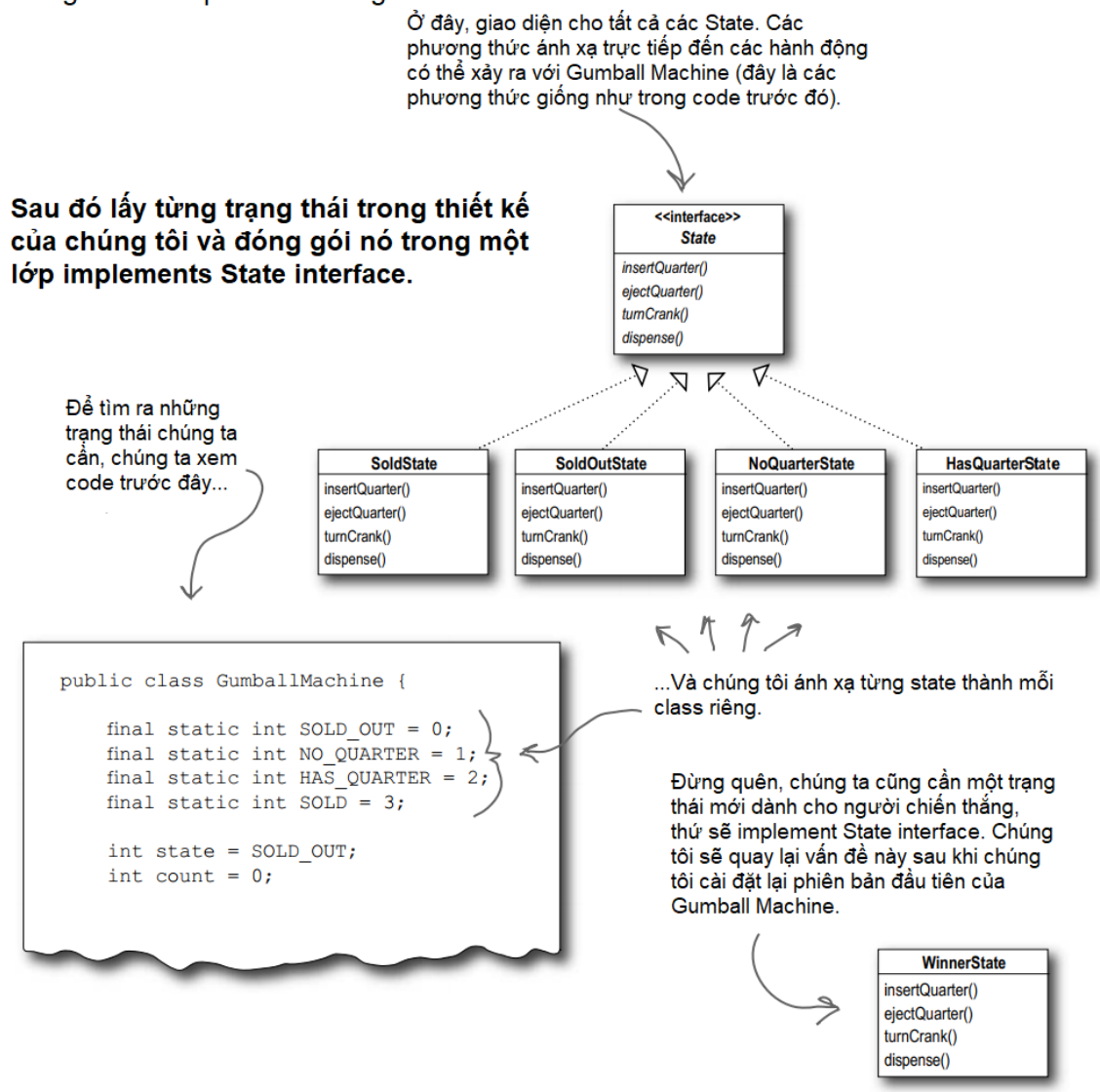
1. Đầu tiên, chúng tôi sẽ định nghĩa một State interface có chứa một phương thức cho mọi hành động trong Gumball Machine.
2. Sau đó, chúng tôi sẽ triển khai một lớp State cho từng trạng thái của máy. Các lớp này sẽ chịu trách nhiệm cho hành vi của máy khi nó ở trạng thái tương ứng.
3. Cuối cùng, chúng tôi sẽ loại bỏ tất cả các code điều kiện (if) và thay vào đó ủy thác cho lớp State để thực hiện công việc cho chúng tôi.

Không chỉ chúng tôi tuân theo các nguyên tắc thiết kế, như bạn thấy, chúng tôi thực sự đang triển khai State Pattern. Nhưng chúng tôi sẽ nhận được tất cả các công cụ State Pattern chính thức sau khi chúng tôi làm lại code của mình…



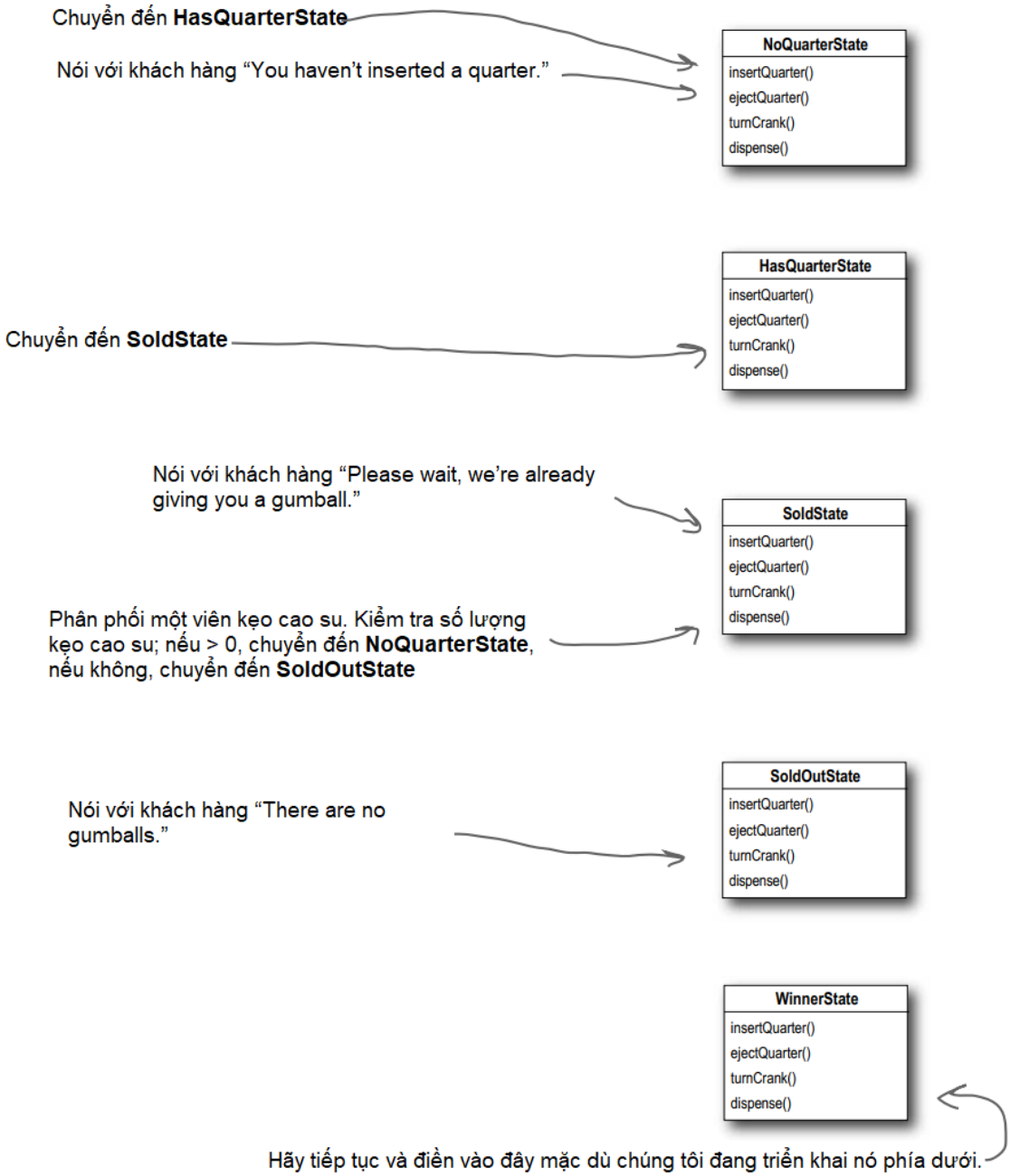
Định nghĩa các interface và class của State Pattern

Trước tiên, hãy tạo ra một interface cho State, tất cả các trạng thái của chúng tôi đều implement chúng:

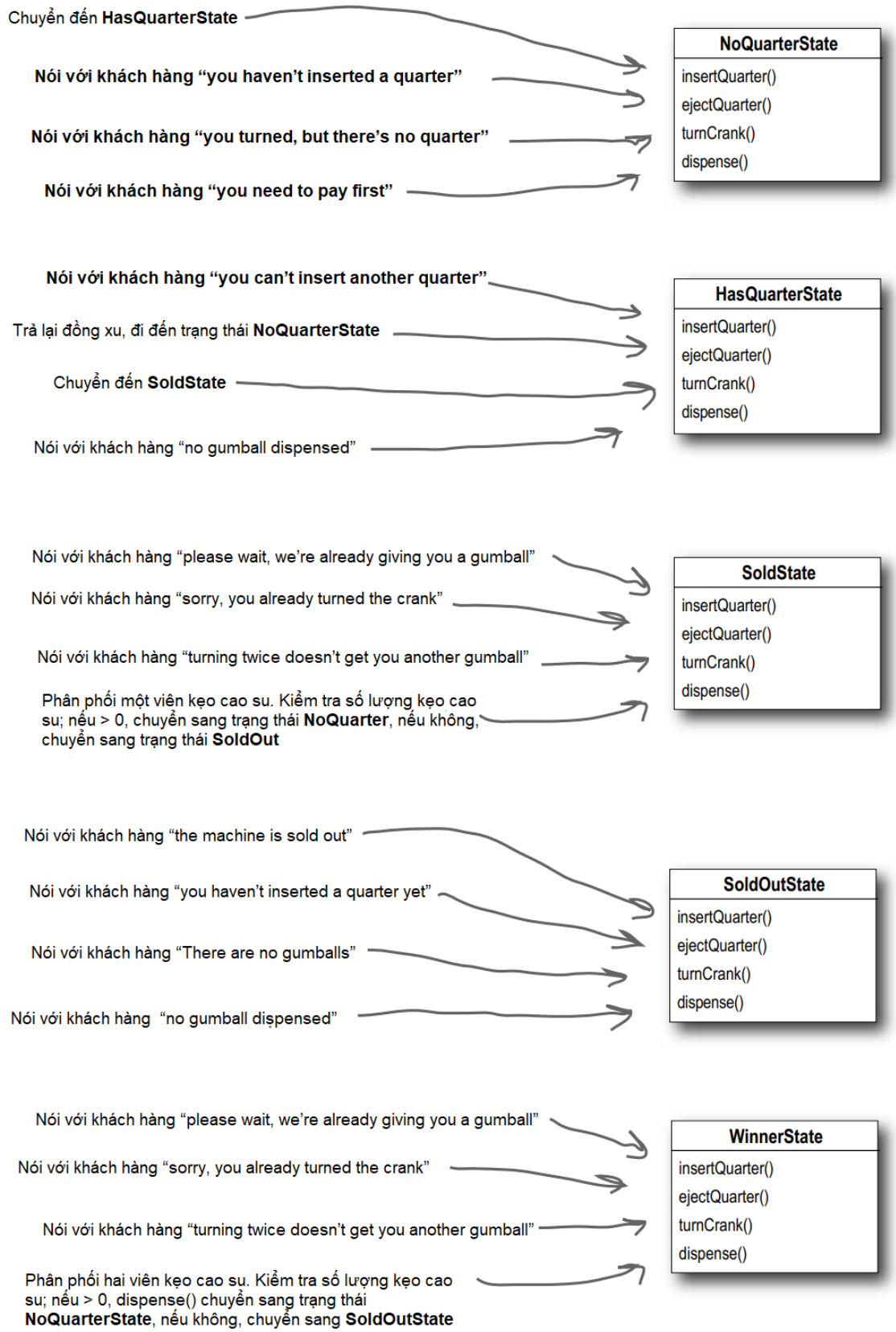


Bài tập:

Để thực hiện các trạng thái của chúng ta, trước tiên chúng ta cần chỉ định hành vi của các lớp khi mỗi hành động được gọi. Chú thích sơ đồ bên dưới với hành vi của từng hành động trong mỗi lớp; chúng tôi đã điền trước vào một vài thứ cho bạn.


Chuyển đến HasQuarterState
Nói với khách hàng "You haven't inserted a quarter."
NoQuarterState
insertQuarter()
ejectQuarter()
turnCrank()
dispense()
HasQuarterState
insertQuarter()
ejectQuarter()
turnCrank()
dispense()
Chuyển đến SoldState
Nói với khách hàng "Please wait, we're already giving you a gumball."
SoldState
insertQuarter()
ejectQuarter()
turnCrank()
dispense()
Phân phối một viên kẹo cao su. Kiểm tra số lượng kẹo cao su; nếu > 0, chuyển đến NoQuarterState, nếu không, chuyển đến SoldOutState
Nói với khách hàng "There are no gumballs."
SoldOutState
insertQuarter()
ejectQuarter()
turnCrank()
dispense()
WinnerState
insertQuarter()
ejectQuarter()
turnCrank()
dispense()
Hãy tiếp tục và điền vào đây mặc dù chúng tôi đang triển khai nó phía dưới.

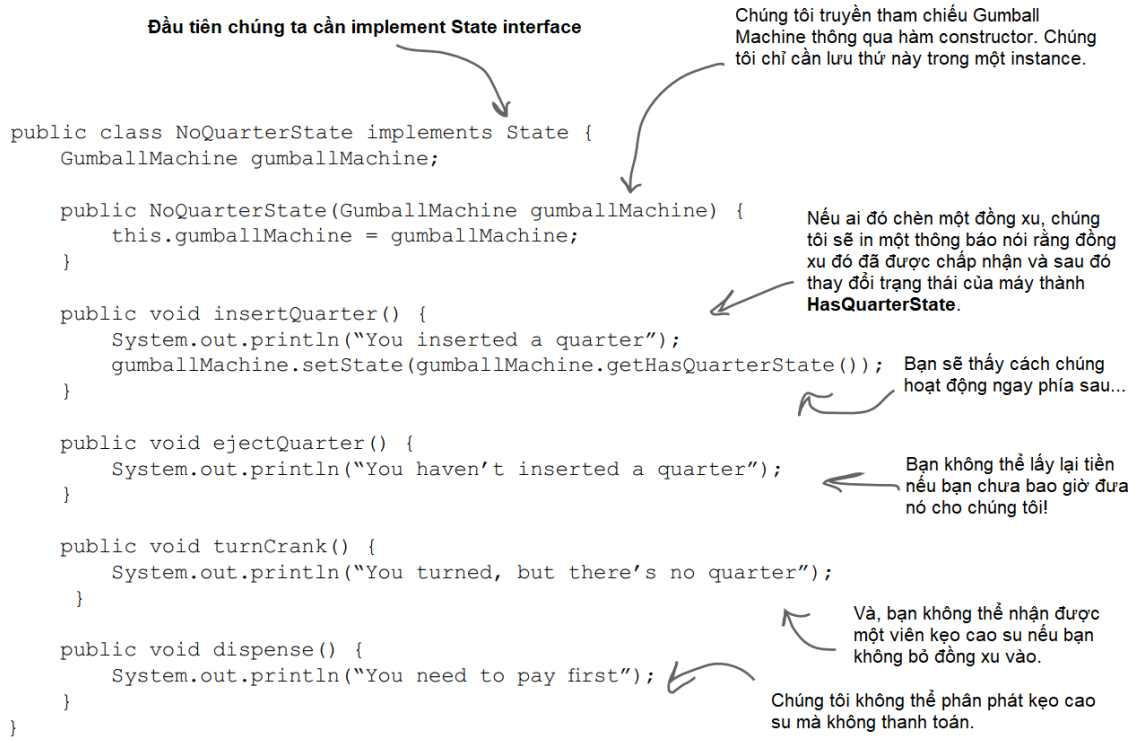
Đáp án:



Implementing State classes của chúng tôi

Thời gian để thực hiện một trạng thái: chúng ta biết những hành vi chúng ta muốn; chúng ta chỉ cần viết nó xuống trong code. Chúng tôi sẽ theo sát code state của máy mà chúng tôi đã viết, nhưng lần này mọi thứ được chia thành các lớp khác nhau.

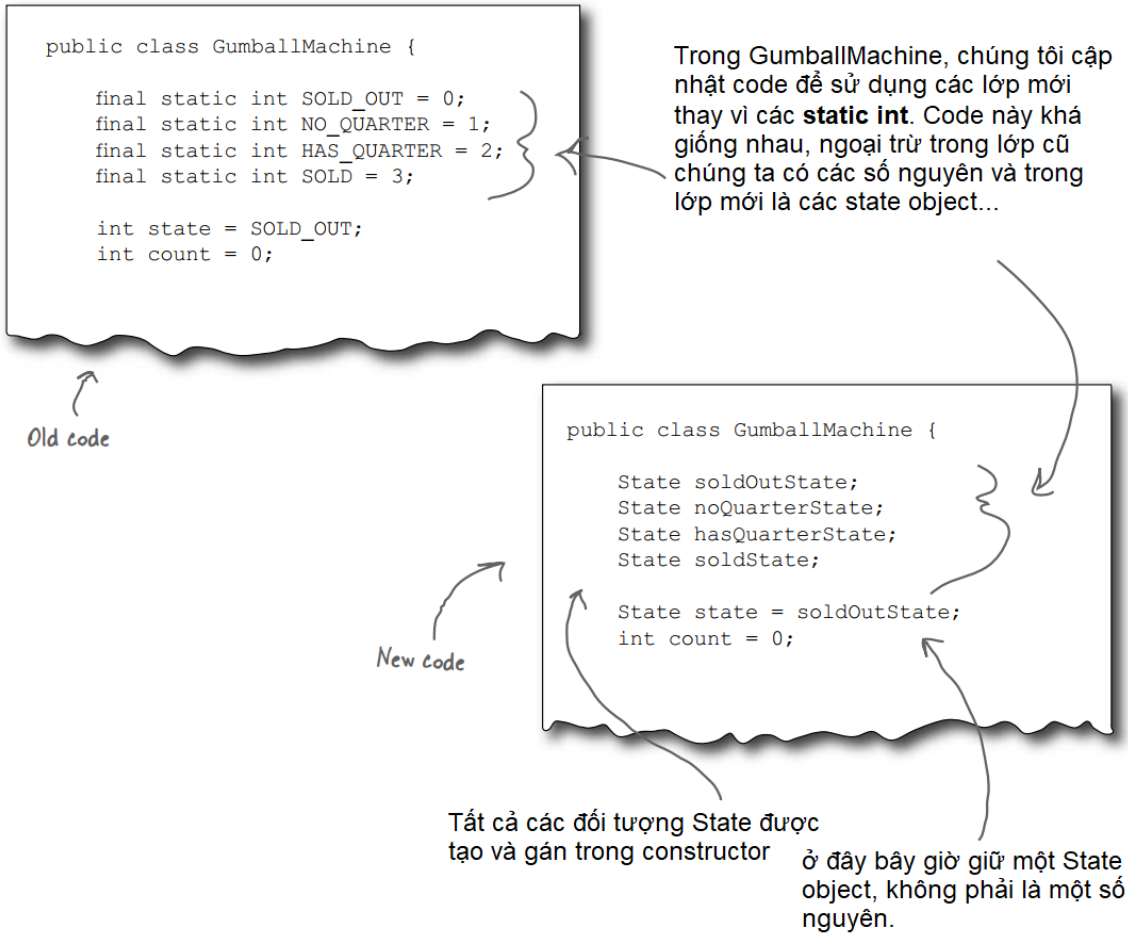
Hãy bắt đầu với **NoQuarterState**:



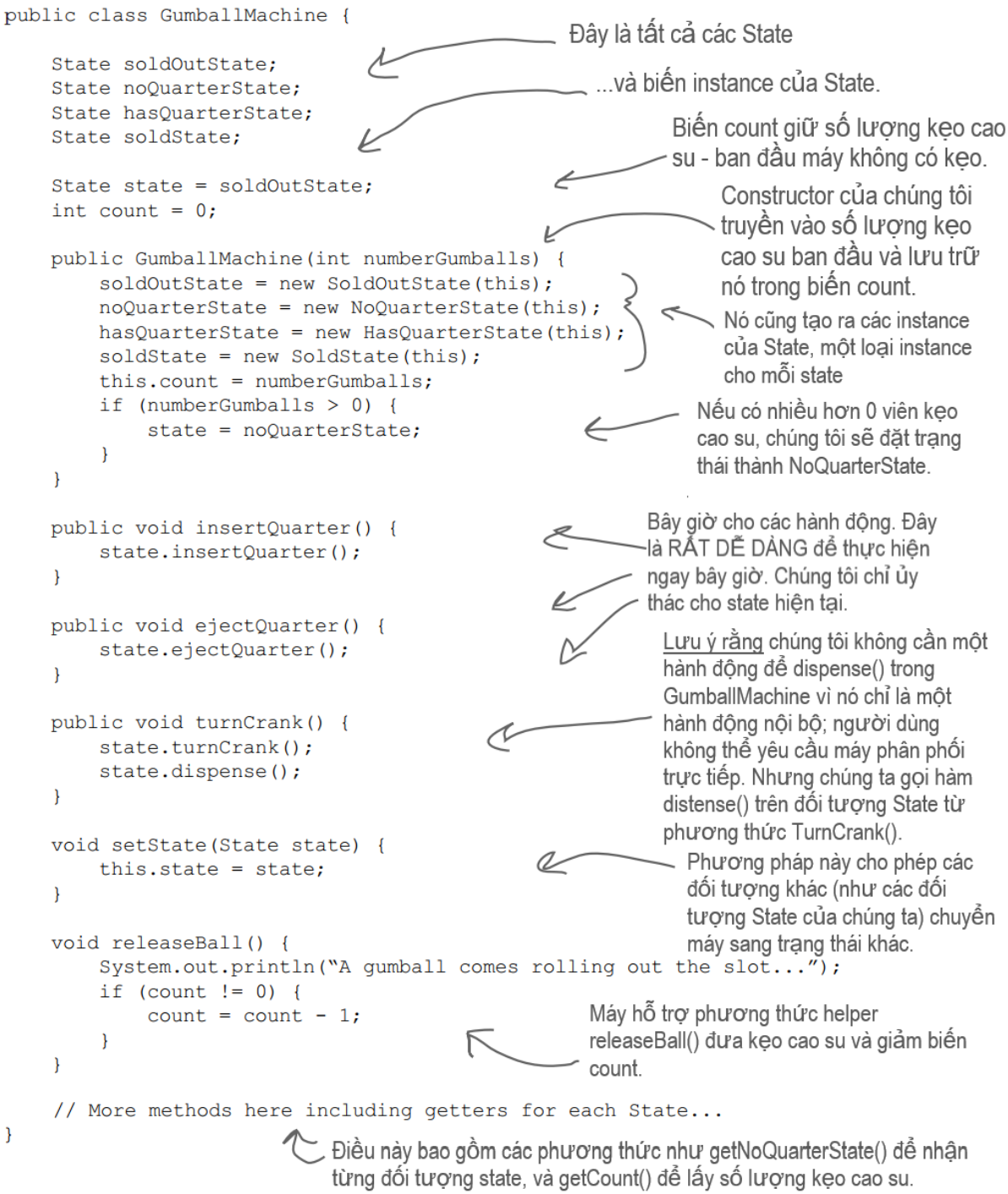
Những gì chúng tôi đang thực hiện là cài đặt các hành vi phù hợp với trạng thái mà chúng tôi đang sử dụng. Trong một số trường hợp, hành vi này bao gồm chuyển Gumball Machine sang trạng thái mới.

Làm lại Gumball Machine với State pattern

Trước khi chúng tôi hoàn thành các lớp State, chúng tôi sẽ làm lại Gumball Machine – theo cách đó bạn có thể thấy tất cả trùng khớp với nhau như thế nào. Chúng tôi sẽ bắt đầu với các biến đối tượng liên quan đến trạng thái và chuyển code từ sử dụng số kiểu int sang sử dụng các state object:



Bây giờ, hãy nhìn vào lớp GumballMachine hoàn chỉnh...




```
public class GumballMachine {

    State soldOutState;
    State noQuarterState;
    State hasQuarterState;
    State soldState;

    State state = soldOutState;
    int count = 0;

    public GumballMachine(int numberGumballs) {
        soldOutState = new SoldOutState(this);
        noQuarterState = new NoQuarterState(this);
        hasQuarterState = new HasQuarterState(this);
        soldState = new SoldState(this);
        this.count = numberGumballs;
        if (numberGumballs > 0) {
            state = noQuarterState;
        }
    }

    public void insertQuarter() {
        state.insertQuarter();
    }

    public void ejectQuarter() {
        state.ejectQuarter();
    }

    public void turnCrank() {
        state.turnCrank();
        state.dispense();
    }

    void setState(State state) {
        this.state = state;
    }

    void releaseBall() {
        System.out.println("A gumball comes rolling out the slot...");
        if (count != 0) {
            count = count - 1;
        }
    }

    // More methods here including getters for each State...
}
```

Thực hiện nhiều State hơn

Bây giờ, khi bạn bắt đầu cảm nhận được cách Gumball Machine và các trạng thái khớp với nhau, hãy cài đặt các lớp **HasQuarterState** và **SoldState** …

```
public class HasQuarterState implements State {
    GumballMachine gumballMachine;

    public HasQuarterState(GumballMachine gumballMachine) {
        this.gumballMachine = gumballMachine;
    }

    public void insertQuarter() {
        System.out.println("You can't insert another quarter");
    }

    public void ejectQuarter() {
        System.out.println("Quarter returned");
        gumballMachine.setState(gumballMachine.getNoQuarterState());
    }

    public void turnCrank() {
        System.out.println("You turned...");
        gumballMachine.setState(gumballMachine.getSoldState());
    }
    public void dispense() {
        System.out.println("No gumball dispensed");
    }
}
```

Khi state được khởi tạo, chúng ta chuyển tham chiếu đến GumballMachine. Điều này được sử dụng để chuyển đổi máy sang các trạng thái khác nhau.

Một hành động không phù hợp cho state này.

Trả lại xu của khách hàng và chuyển trở lại NoQuarterState.

Khi quay tay quay, chúng ta chuyển máy sang trạng thái SoldState bằng cách gọi phương thức setState() của nó và truyền cho nó đối tượng SoldState. Đối tượng SoldState được truy xuất bằng phương thức getter getSoldState() (mỗi trạng thái có một trong các phương thức getter này).

Một hành động không phù hợp khác cho trạng thái này.

Bây giờ, hãy kiểm tra lớp **SoldState**…

```
public class SoldState implements State {
    //constructor and instance variables here

    public void insertQuarter() {
        System.out.println("Please wait, we're already giving you a gumball");
    }

    public void ejectQuarter() {
        System.out.println("Sorry, you already turned the crank");
    }

    public void turnCrank() {
        System.out.println("Turning twice doesn't get you another gumball!");
    }

    public void dispense() {
        gumballMachine.releaseBall();
        if (gumballMachine.getCount() > 0) {
            gumballMachine.setState(gumballMachine.getNoQuarterState());
        } else {
            System.out.println("Oops, out of gumballs!");
            gumballMachine.setState(gumballMachine.getSoldOutState());
        }
    }
}
```

Dưới đây là tất cả các hành động không phù hợp cho trạng thái này

Và ở đây, nơi công việc thực sự được bắt đầu...

Chúng tôi tích hợp trong **SoldState**, có nghĩa là khách hàng đã trả tiền. Vì vậy, trước tiên chúng ta cần yêu cầu máy phát hành một viên kẹo cao su.

Sau đó, chúng tôi kiểm tra số kẹo còn lại và chuyển sang **NoQuarterState** hoặc **SoldOutState**.

Sử dụng sức mạnh bộ não

Nhìn lại việc thực hiện **GumballMachine**. Nếu tay quay được quay và không thành công (giả sử khách hàng đã không bỏ vào một đồng xu trước), dù sao chúng tôi cũng gọi là "phân phối", mặc dù điều đó không cần thiết. Làm thế nào bạn có thể khắc phục điều này?

Chúng tôi có một lớp còn lại mà chúng tôi đã thực hiện: **SoldOutState**. Tại sao bạn không thực hiện nó? Để làm điều này, hãy suy nghĩ cẩn thận về cách Gumball Machine nên hành xử trong từng tình huống. Kiểm tra câu trả lời của bạn trước khi tiếp tục…

```
public class SoldOutState implements [          ] {
    GumballMachine gumballMachine;

    public SoldOutState(GumballMachine gumballMachine) {


    }

    public void insertQuarter() {



    }

    public void ejectQuarter() {



    }

    public void turnCrank() {



    }

    public void dispense() {



    }

}
```

Đáp án:

In the Sold Out state, we really can't do anything until someone refills the Gumball Machine.

```
public class SoldOutState implements State {
    GumballMachine gumballMachine;

    public SoldOutState(GumballMachine gumballMachine) {
        this.gumballMachine = gumballMachine;
    }

    public void insertQuarter() {
        System.out.println("You can't insert a quarter, the machine is sold out");
    }

    public void ejectQuarter() {
        System.out.println("You can't eject, you haven't inserted a quarter yet");
    }

    public void turnCrank() {
        System.out.println("You turned, but there are no gumballs");
    }

    public void dispense() {
        System.out.println("No gumball dispensed");
    }

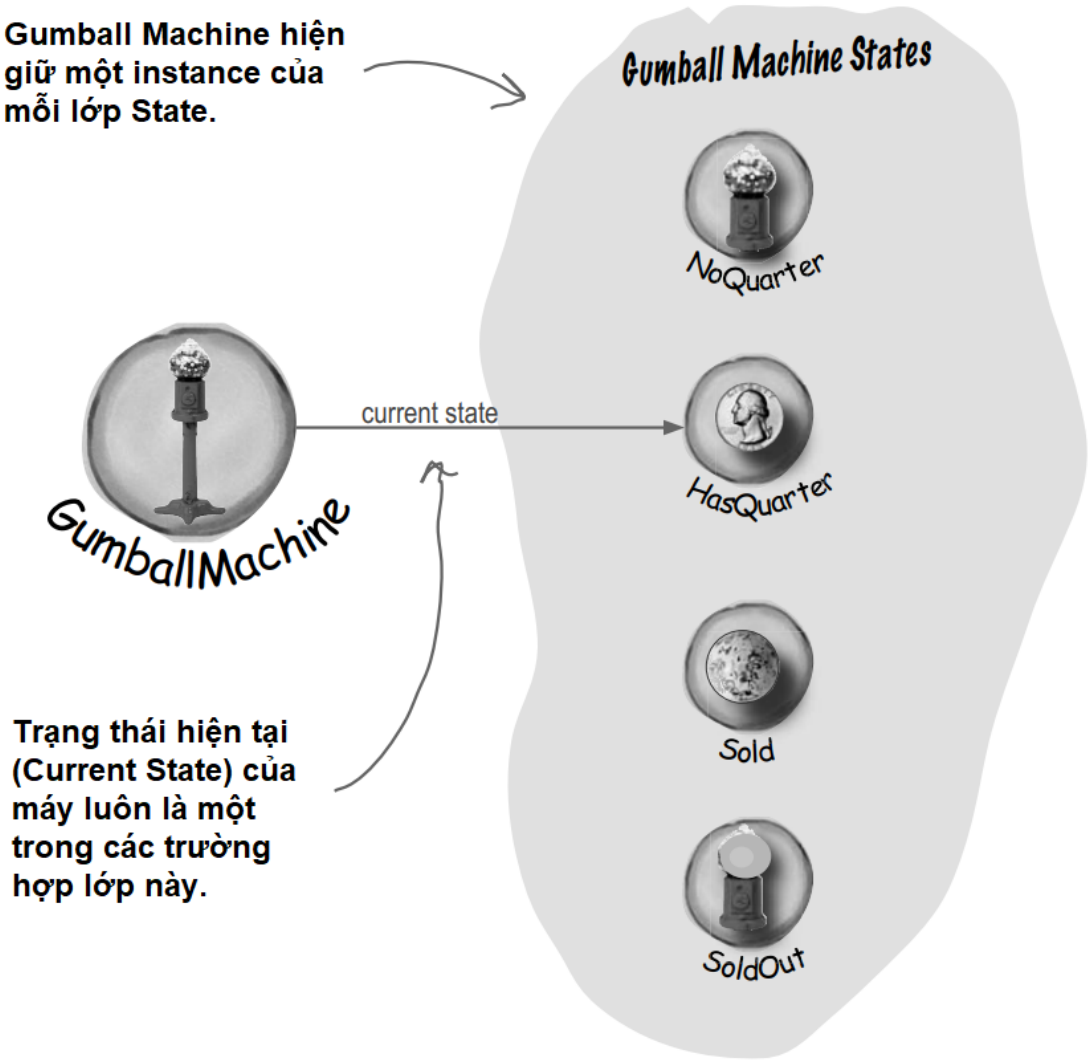
}
```

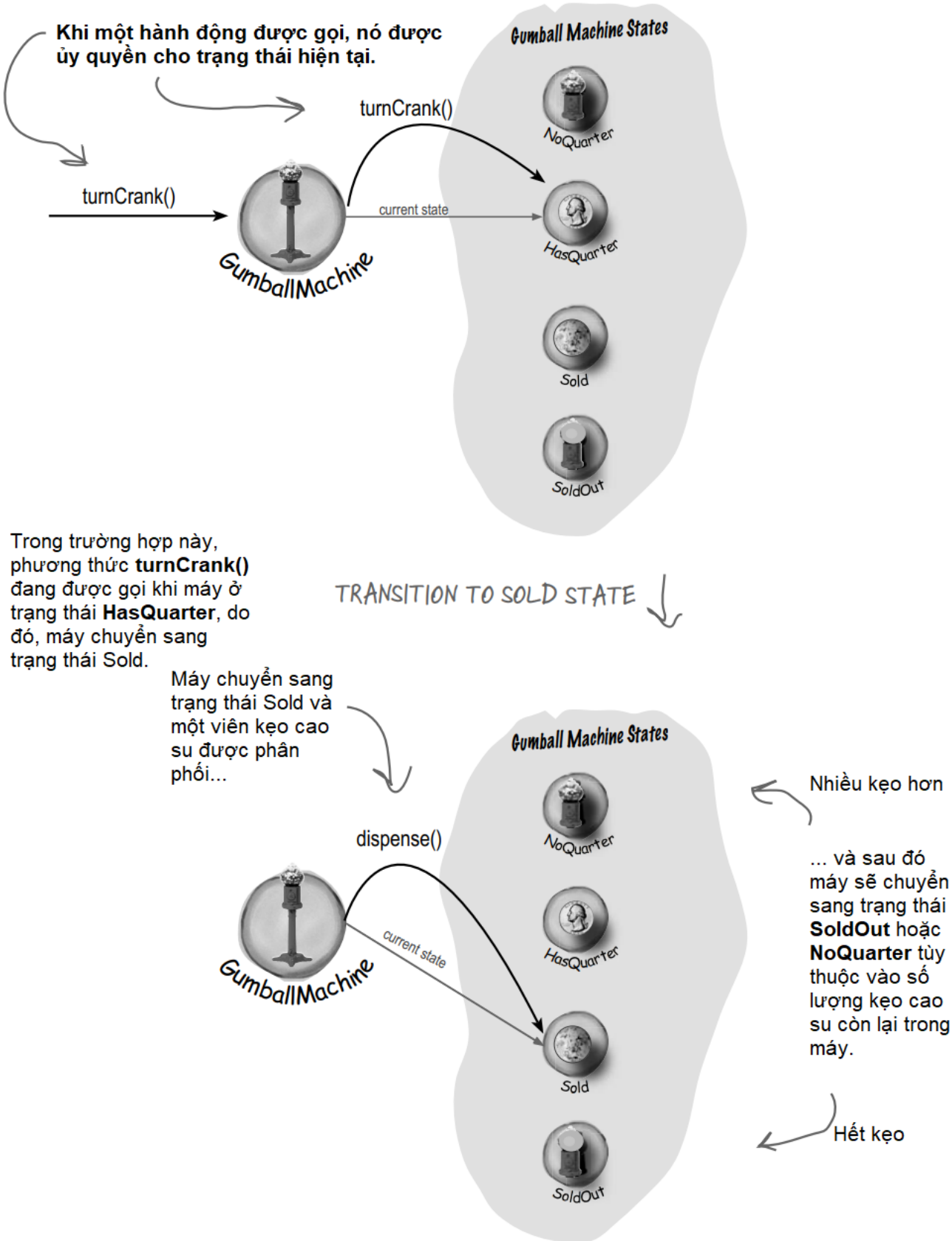
Chúng ta hãy nhìn vào những gì chúng ta đã làm cho đến giờ...

Để bắt đầu, bây giờ bạn có một triển khai Gumball Machine có cấu trúc khá khác so với phiên bản đầu tiên của bạn, và về mặt chức năng thì nó hoàn toàn giống nhau. Bằng cách thay đổi cấu trúc việc triển khai, bạn phải:

- "Địa phương hóa" hành vi của mỗi trạng thái vào lớp riêng của chúng.
- Loại bỏ tất cả những rắc rối về câu lệnh if, thứ mà sẽ khó bảo trì.
- Đã đóng từng trạng thái để sửa đổi và vẫn để Gumball Machine mở rộng bằng cách thêm các lớp trạng thái mới (và chúng tôi sẽ làm điều này ngay phía sau).
- Tạo một code base và cấu trúc lớp ánh xạ gần hơn với sơ đồ Mighty Gumball và dễ đọc, dễ hiểu hơn.

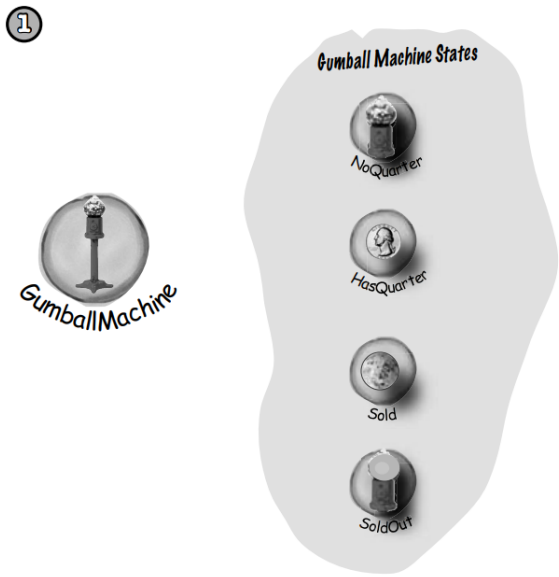
Bây giờ, hãy nhìn sâu hơn về khía cạnh chức năng của những gì chúng ta đã làm:

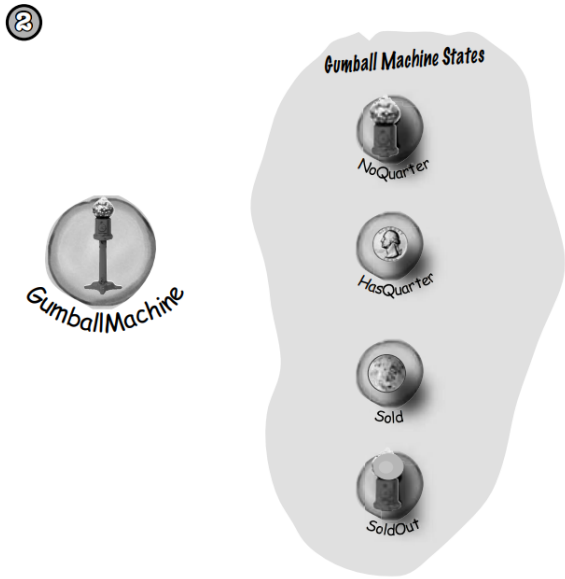

Gumball Machine hiện giữ một instance của mỗi lớp State.
Gumball Machine States
NoQuarter
current state
HasQuarter
GumballMachine
Trạng thái hiện tại (Current State) của máy luôn là một trong các trường hợp lớp này.
Sold
SoldOut

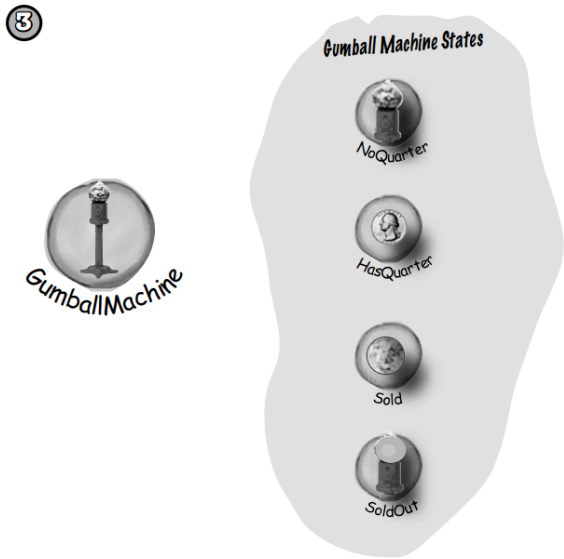


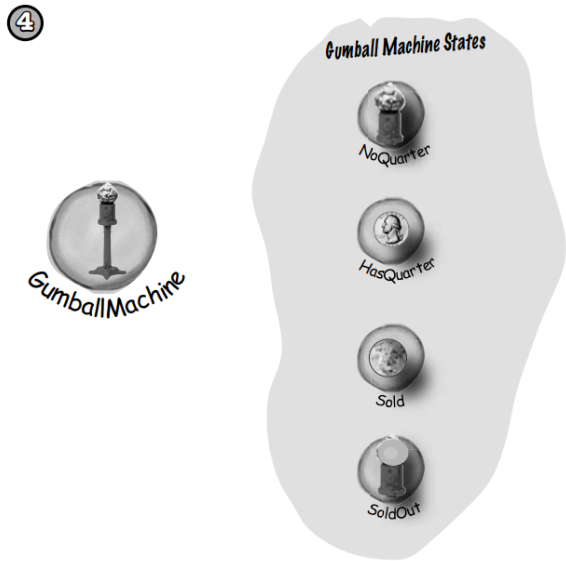
Hậu trường State pattern: Chuyến tham quan tự túc

Bài tập: Theo dõi các bước của Gumball Machine bắt đầu với trạng thái **NoQuarter**. Cũng chú thích sơ đồ với các hành động và đầu ra của máy. Đối với bài tập này, bạn có thể cho rằng có rất nhiều kẹo cao su trong máy.

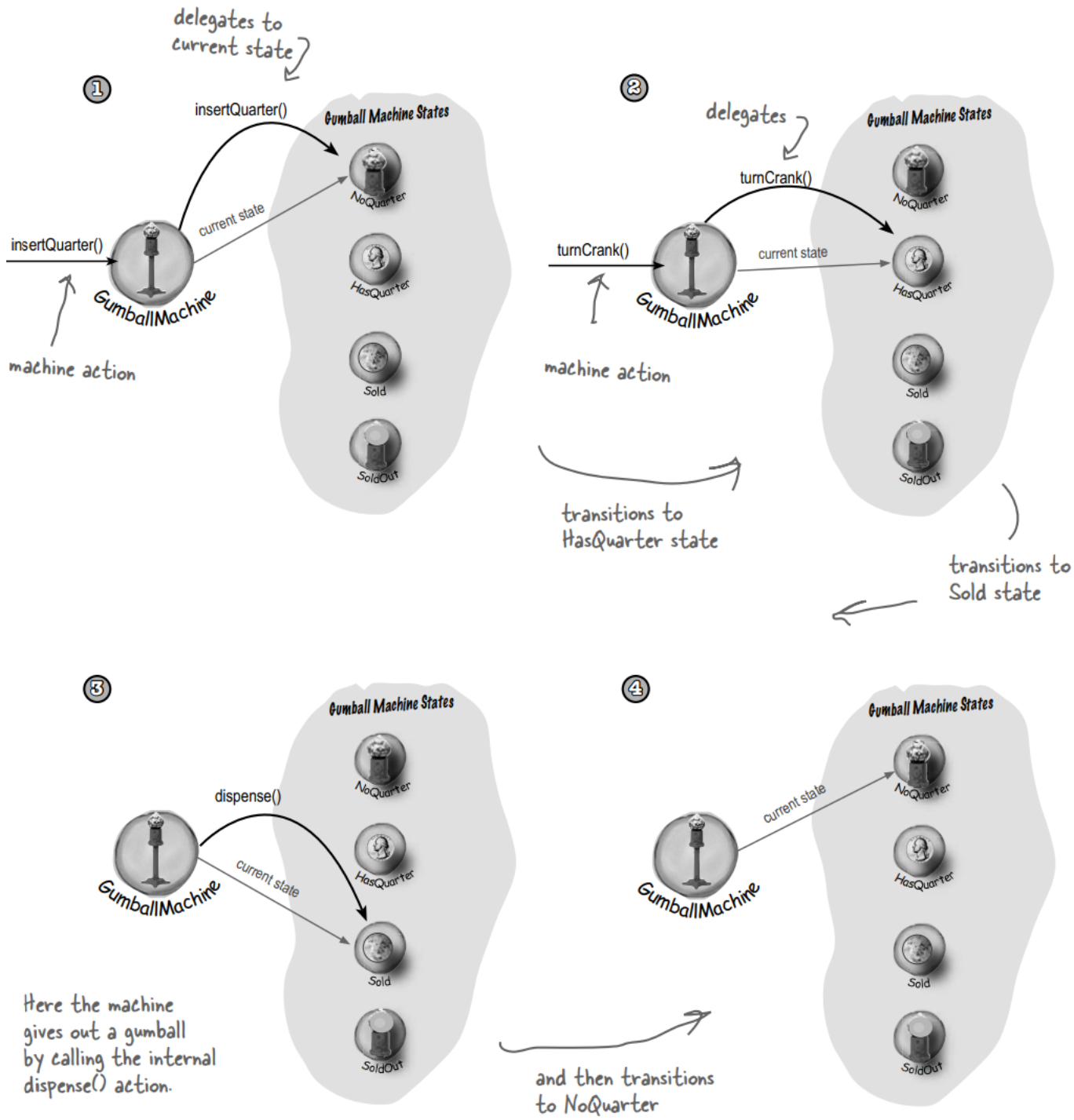

1
GumballMachine
Gumball Machine States
NoQuarter
HasQuarter
Sold
SoldOut


2
GumballMachine
Gumball Machine States
NoQuarter
HasQuarter
Sold
SoldOut


3
GumballMachine
Gumball Machine States
NoQuarter
HasQuarter
Sold
SoldOut


4
GumballMachine
Gumball Machine States
NoQuarter
HasQuarter
Sold
SoldOut

Đáp án:



Định nghĩa State Pattern

Vâng, đó là sự thật, chúng tôi chỉ thực hiện State Pattern. Vì vậy, bây giờ, hãy xem qua tất cả những gì về nó:

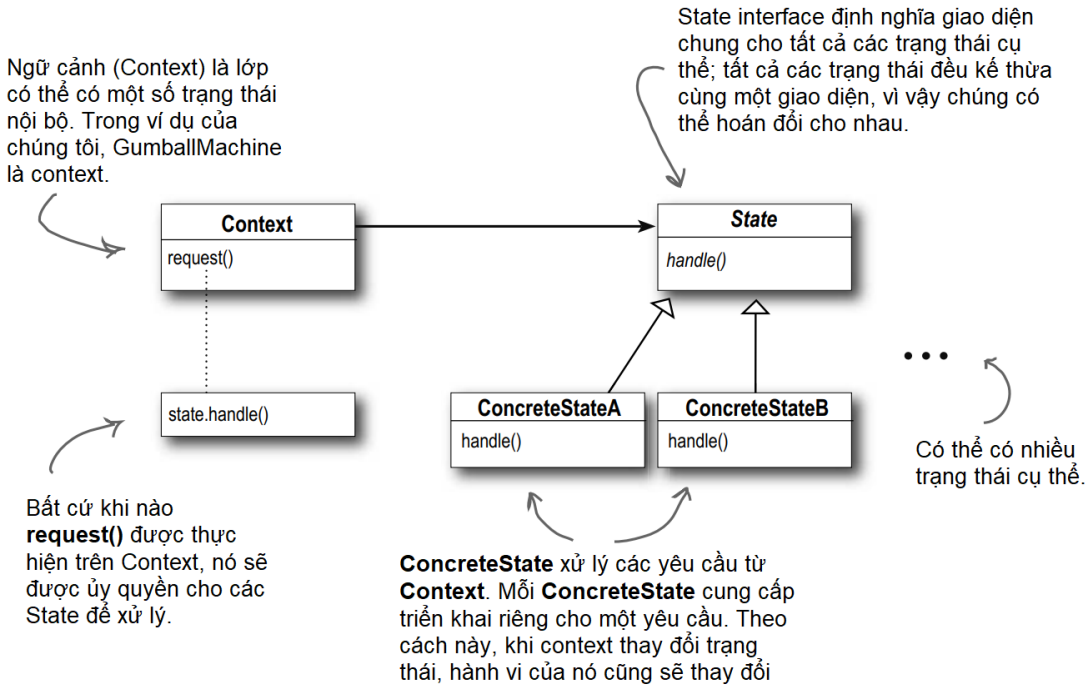
> **The State Pattern** allows an object to alter its behavior when its internal state changes. The object will appear to change its class.

(State Pattern cho phép một đối tượng thay đổi hành vi của nó khi trạng thái bên trong của nó thay đổi. Đối tượng sẽ xuất hiện để thay đổi lớp của nó)

Phần đầu tiên của định nghĩa này rất có ý nghĩa, phải không? Bởi vì mẫu này đóng gói trạng thái thành các lớp riêng biệt và ủy quyền cho đối tượng đại diện cho trạng thái hiện tại, chúng tôi biết rằng hành vi thay đổi cùng với trạng thái bên trong. Gumball Machine cung cấp một ví dụ hay: khi máy Gumball ở trạng thái **NoQuarterState** và bạn bỏ vào một đồng xu, bạn sẽ có hành vi khác (máy chấp nhận một đồng xu) và khi bạn bỏ vào một đồng xu vào khi máy ở trạng thái **HasQuarterState** (máy từ chối đồng xu).

Còn phần thứ hai của định nghĩa thì sao? Điều đó có ý nghĩa gì đối với một đối tượng có vẻ như thay đổi lớp của nó? Hãy nghĩ về nó từ góc độ của một khách hàng: "nếu một đối tượng bạn đang sử dụng có thể thay đổi hoàn toàn hành vi của nó, thì đối tượng đó thực sự được khởi tạo từ một lớp khác". Tuy nhiên, trong thực tế, bạn biết rằng chúng ta đang sử dụng composition để tạo ra sự thay đổi của lớp bằng cách tham chiếu các state object khác nhau.

Sơ đồ lớp State pattern



Sơ đồ lớp State pattern



Sơ đồ lớp State pattern hơi giống Strategy Pattern (https://toihocdesignpattern.com/chuong-1-strategy-pattern-chao-mung-den-voi-design-patterns.html)

Bạn đã có một quan sát tốt! Vâng, các sơ đồ lớp về cơ bản là giống nhau, nhưng hai mẫu này khác nhau trong *ý định* của chúng.

Với **State Pattern**, chúng ta có một tập hợp các hành vi được đóng gói trong các state object; bất cứ lúc nào cần thiết, **Context** sẽ ủy thác cho một trong những trạng thái đó. Theo thời gian, trạng thái hiện tại thay đổi trên từng state object để phản ánh trạng thái của Context, do đó, hành vi của Context cũng thay đổi theo thời gian. Client thường biết rất ít về các state object.

Với Strategy Pattern (https://toihocdesignpattern.com/chuong-1-strategy-pattern-chao-mung-den-voi-design-patterns.html), client thường chỉ định strategy object mà Context mong muốn. Bây giờ, trong khi mẫu cung cấp tính linh hoạt để thay đổi strategy object trong runtime, thường có một strategy object phù hợp nhất cho một Context object. Ví dụ, trong Chương 1 (https://toihocdesignpattern.com/chuong-1-strategy-pattern-chao-mung-den-voi-design-patterns.html), một số con vịt của chúng tôi được thiết đặt để bay với hành vi bay bình thường (như vịt trời), trong khi những con khác được thiết đặt với hành vi bay giữ cho chúng dưới đất (như vịt cao su).

Nói chung, hãy nghĩ về Strategy Pattern (https://toihocdesignpattern.com/chuong-1-strategy-pattern-chao-mung-den-voi-design-patterns.html) như một sự thay thế linh hoạt cho các lớp con; nếu bạn sử dụng tính kế thừa để xác định hành vi của một lớp, thì bạn đã bị mắc kẹt với hành vi đó ngay cả khi bạn cần thay đổi nó. Với Strategy, bạn có thể thay đổi hành vi bằng cách kết hợp với một đối tượng khác.

Hãy nghĩ về **State Pattern** như một giải pháp thay thế cho việc đưa nhiều điều kiện vào ngữ cảnh của bạn; bằng cách gói gọn các hành vi trong các State object, bạn có thể chỉ cần thay đổi đối tượng trạng thái trong ngữ cảnh để thay đổi hành vi của nó.

Không có câu hỏi ngớ ngẩn

Hỏi: Trong **GumballMachine**, các state quyết định trạng thái tiếp theo sẽ là gì. **ConcreteState** có luôn quyết định đâu sẽ là trạng thái tiếp theo hay không?

Đáp: Không, không phải lúc nào cũng vậy. Có một lựa chọn khác là để cho **Context** quyết định luồng chuyển đổi trạng thái.

Theo nguyên tắc chung, khi các chuyển đổi trạng thái là cố định, chúng thích hợp để đưa vào **Context** hơn; tuy nhiên, khi quá trình chuyển đổi cần tính linh hoạt, chúng thường được đặt trong chính các state class (ví dụ, trong **GumballMachine**, sự lựa chọn chuyển đổi sang **NoQuarter** hoặc **SoldOut** phụ thuộc vào số kẹo cao su còn lại).

Nhược điểm của việc chuyển trạng thái trong các state class là chúng ta tạo ra sự phụ thuộc giữa các lớp trạng thái. Khi triển khai **GumballMachine**, chúng tôi đã cố gắng giảm thiểu điều này bằng cách sử dụng các phương thức getter trong **Context**, thay vì code trên các lớp **ConcreteState**.

Lưu ý rằng bằng cách đưa ra quyết định này, bạn đang đưa ra quyết định về việc các lớp nào được đóng để sửa đổi – **Context** hoặc các **State class** – khi hệ thống phát triển.

Hỏi: Client có bao giờ tương tác trực tiếp với các trạng thái không?

Đáp: Không. Các trạng thái được Context sử dụng để thể hiện trạng thái và hành vi bên trong của nó, vì vậy tất cả các yêu cầu đối với các State object đều đến từ Context. Client không trực tiếp thay đổi trạng thái của Context. Đây là công việc theo ngữ cảnh để giám sát trạng thái của nó và bạn không muốn một client thay đổi trạng thái của một context khi chúng không hề hay biết.

Hỏi: Nếu tôi có nhiều instance của Context trong ứng dụng của mình, tôi có thể chia sẻ các state object giữa chúng không?

Đáp: Vâng, hoàn toàn, và trên thực tế đây là một kịch bản rất phổ biến. Yêu cầu duy nhất là các đối tượng trạng thái của bạn không giữ trạng thái nội bộ của riêng chúng; nếu không, bạn sẽ cần một phiên bản duy nhất cho mỗi ngữ cảnh.

Để chia sẻ trạng thái, bạn sẽ gán mỗi trạng thái cho một biến đối tượng tĩnh. Nếu trạng thái của bạn cần sử dụng các phương thức hoặc biến đối tượng trong **Context** của bạn, thì bạn cũng sẽ phải cung cấp cho nó một tham chiếu đến **Context** trong mỗi phương thức **handler()**.

Hỏi: Có vẻ như việc sử dụng State Pattern luôn làm tăng số lượng các lớp trong thiết kế của chúng tôi. Hãy xem **GumballMachine** của chúng tôi tăng bao nhiêu lớp so với thiết kế ban đầu!

Đáp: Bạn nói đúng, bằng cách gói hành vi trạng thái vào các lớp trạng thái riêng biệt, bạn sẽ luôn có nhiều lớp hơn trong thiết kế của mình. Thường là cái giá bạn phải trả cho sự linh hoạt. Trừ khi code của bạn chỉ cần thiết kế và cài đặt một lần duy nhất và không sẽ không bao giờ thay đổi, hãy xem xét việc xây dựng nó với các "lớp bổ sung" và bạn có thể cảm ơn chính mình (khi bảo trì). Lưu ý rằng thường thì điều quan trọng là số lượng các lớp bạn giao tiếp với client của mình và có nhiều cách để ẩn các "lớp bổ sung" này khỏi client của bạn (giả sử bằng cách khai báo private, protected…).

Ngoài ra, hãy xem xét lựa chọn thay thế: nếu bạn có một ứng dụng có nhiều trạng thái và bạn quyết định không sử dụng các đối tượng riêng biệt, thay vào đó, bạn sẽ kết thúc bằng các câu lệnh **If**. Điều này làm cho code của bạn khó duy trì và có thể hiểu. Bằng cách sử dụng các object, bạn làm cho các trạng thái rõ ràng và dễ hiểu và duy trì code.

Hỏi: Sơ đồ lớp State Pattern cho thấy State là một lớp trừu tượng. Nhưng không phải bạn đã sử dụng một interface trong việc cài đặt trạng thái của gumball machine hay sao?

Đáp: Vâng. Do chúng tôi không có chức năng chung để đưa vào một lớp trừu tượng, chúng tôi làm nó với một interface. Trong triển khai của riêng bạn, bạn có thể muốn xem xét đến một lớp trừu tượng. Làm như vậy có lợi ích cho phép bạn thêm các phương thức vào lớp trừu tượng sau này, mà không phá vỡ các concrete state.

Chúng tôi vẫn cần hoàn thành trò chơi "Gumball 1 phần 10"

Hãy nhớ rằng, chúng ta chưa hoàn thành. Chúng ta còn có một trò chơi để cài đặt; nhưng bây giờ chúng ta đã thực hiện State Pattern, nó sẽ rất dễ dàng. Đầu tiên, chúng ta cần thêm một trạng thái vào lớp **GumballMachine**:

```
public class GumballMachine {

    State soldOutState;
    State noQuarterState;
    State hasQuarterState;
    State soldState;
    State winnerState;

    State state = soldOutState;
    int count = 0;

    // methods here
}
```

Tất cả những gì bạn cần thêm ở đây là **WinnerState** mới và khởi tạo nó trong hàm constructor.

Đừng quên bạn cũng phải thêm một phương thức getter cho **WinnerState** nữa.

Bây giờ, hãy để cho bản thân lớp **WinnerState** thực hiện các hành động, nó giống với lớp **SoldState**:

```
public class WinnerState implements State {

    // instance variables and constructor

    // insertQuarter error message

    // ejectQuarter error message

    // turnCrank error message

    public void dispense() {
        System.out.println("YOU'RE A WINNER! You get two gumballs for your quarter");
        gumballMachine.releaseBall();
        if (gumballMachine.getCount() == 0) {
            gumballMachine.setState(gumballMachine.getSoldOutState());
        } else {
            gumballMachine.releaseBall();
            if (gumballMachine.getCount() > 0) {
                gumballMachine.setState(gumballMachine.getNoQuarterState());
            } else {
                System.out.println("Oops, out of gumballs!");
                gumballMachine.setState(gumballMachine.getSoldOutState());
            }
        }
    }
}
```

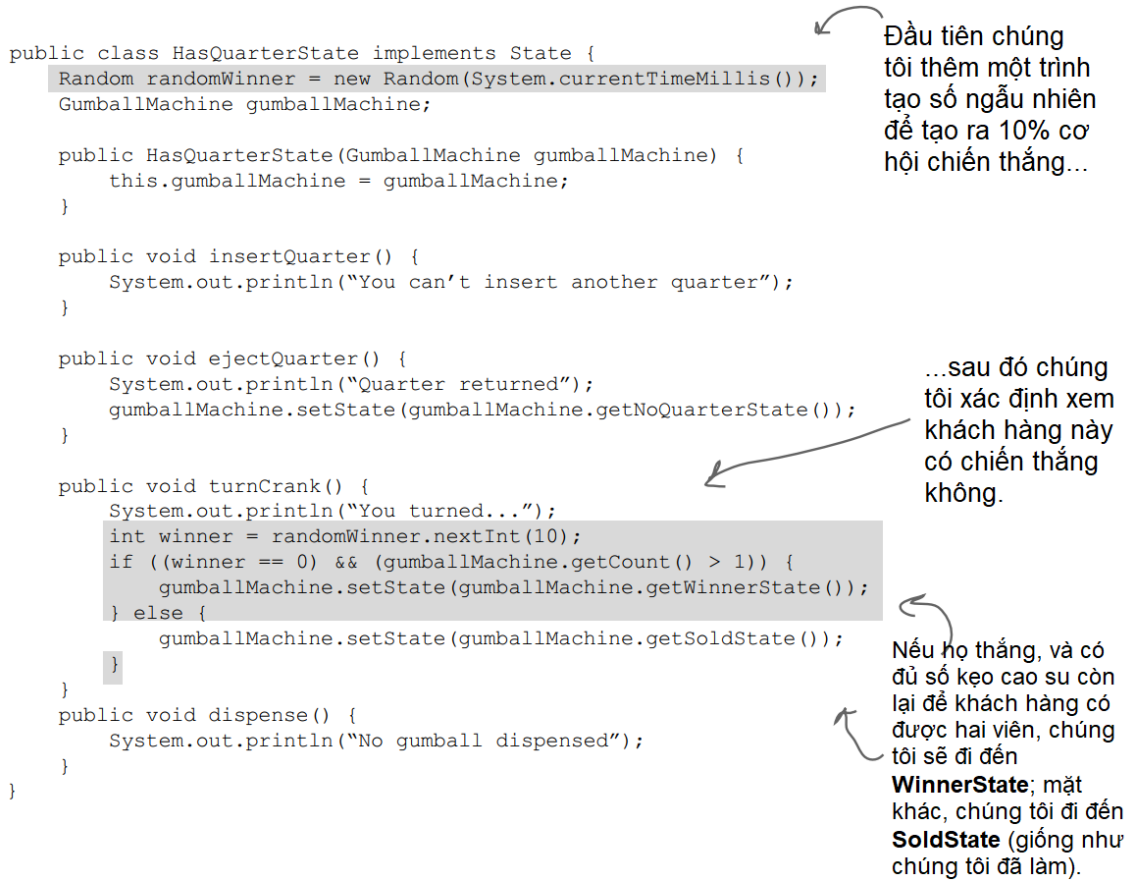
Giống với **SoldState**

Ở đây chúng tôi phát hành hai viên kẹo cao su và sau đó chuyển đến **NoQuarterState** hoặc **SoldOutState**.

Miễn là chúng tôi có một viên kẹo cao su thứ hai, chúng tôi phân phối nó.

Hoàn thành trò chơi

Chúng ta chỉ cần thực hiện một thay đổi nữa: chúng ta cần cài đặt "cơ hội ngẫu nhiên" trong trò chơi và thêm một chuyển đổi trạng thái đến **WinnerState**. Chúng tôi sẽ thêm cả hai vào **HasQuarterState** bất cứ khi nào khách hàng xoay tay quay:




```
public class HasQuarterState implements State {
    Random randomWinner = new Random(System.currentTimeMillis());
    GumballMachine gumballMachine;

    public HasQuarterState(GumballMachine gumballMachine) {
        this.gumballMachine = gumballMachine;
    }

    public void insertQuarter() {
        System.out.println("You can't insert another quarter");
    }

    public void ejectQuarter() {
        System.out.println("Quarter returned");
        gumballMachine.setState(gumballMachine.getNoQuarterState());
    }

    public void turnCrank() {
        System.out.println("You turned...");
        int winner = randomWinner.nextInt(10);
        if ((winner == 0) && (gumballMachine.getCount() > 1)) {
            gumballMachine.setState(gumballMachine.getWinnerState());
        } else {
            gumballMachine.setState(gumballMachine.getSoldState());
        }
    }
    public void dispense() {
        System.out.println("No gumball dispensed");
    }
}
```

Wow, điều đó khá đơn giản để thực hiện! Chúng tôi vừa thêm một trạng thái mới vào **GumballMachine** và sau đó cài đặt nó. Tất cả những gì chúng tôi phải làm là thực hiện trò chơi may rủi của mình và chuyển sang trạng thái chính xác. Có vẻ như chiến lược viết code mới của chúng tôi đang được đền đáp…

Demo cho CEO của Mighty Gumball, Inc.

Giám đốc điều hành của Mighty Gumball đã ghé qua để xem giới thiệu về code trò chơi mới được làm lại của bạn. Hãy hy vọng những trạng thái đó chuyển đổi đúng theo suy nghĩ! Chúng tôi sẽ giữ cho bản demo ngắn (khoảng chú ý ngắn của các CEO sẽ là tài liệu tốt).

Code này thực sự đã không thay đổi chút nào; chúng tôi chỉ rút ngắn nó một chút.

Một lần nữa, bắt đầu với một máy bắn kẹo cao su với 5 viên kẹo.

```
public class GumballMachineTestDrive {
    public static void main(String[] args) {
        GumballMachine gumballMachine = new GumballMachine(5);

        System.out.println(gumballMachine);

        gumballMachine.insertQuarter();
        gumballMachine.turnCrank();

        System.out.println(gumballMachine);

        gumballMachine.insertQuarter();
        gumballMachine.turnCrank();
        gumballMachine.insertQuarter();
        gumballMachine.turnCrank();

        System.out.println(gumballMachine);
    }
}
```

Chúng tôi muốn có được một trạng thái chiến thắng, vì vậy chúng tôi chỉ cần tiếp tục đặt đồng xu và xoay tay quay. Chúng tôi in trạng thái của máy bắn kẹo cao su thường xuyên...

Toàn bộ đội ngũ kỹ thuật đang chờ bên ngoài phòng hội thảo để xem liệu thiết kế dựa trên State Pattern mới có hoạt động không !!





Trời ạ, chúng ta đã gặp may hay sao? Trong bản demo cho CEO, chúng tôi đã thắng không chỉ một lần, mà là hai lần!

```
File  Edit  Window  Help  Whenisagumballajawbreaker?
%java GumballMachineTestDrive
Mighty Gumball, Inc.
Java-enabled Standing Gumball Model #2004
Inventory: 5 gumballs
Machine is waiting for quarter

You inserted a quarter
You turned...
YOU'RE A WINNER! You get two gumballs for your quarter
A gumball comes rolling out the slot...
A gumball comes rolling out the slot...

Mighty Gumball, Inc.
Java-enabled Standing Gumball Model #2004
Inventory: 3 gumballs
Machine is waiting for quarter

You inserted a quarter
You turned...
A gumball comes rolling out the slot...
You inserted a quarter
You turned...
YOU'RE A WINNER! You get two gumballs for your quarter
A gumball comes rolling out the slot...
A gumball comes rolling out the slot...
Oops, out of gumballs!

Mighty Gumball, Inc.
Java-enabled Standing Gumball Model #2004
Inventory: 0 gumballs
Machine is sold out
%
```

Không có câu hỏi ngớ ngẩn

Hỏi: Tại sao chúng ta cần **WinnerState**? Chúng ta có thể xử lý trả về hai viên kẹo cao su trong lớp **SoldState** không?

Đáp: Đó là một câu hỏi hay. **SoldState** và **WinnerState** gần như giống hệt nhau, ngoại trừ việc **WinnerState** trả về hai viên kẹo cao su thay vì một. Bạn chắc chắn có thể viết code để đưa đoạn code trả hai viên kẹo cao su vào **SoldState**. Tất nhiên, nhược điểm là bây giờ bạn đã có HAI trạng thái được đại diện trong một lớp State: **trạng thái mà bạn là người chiến thắng** và **trạng thái mà bạn không phải là người chiến thắng**. Vì vậy, bạn đang hy sinh "sự rõ ràng" trong lớp State để giảm trùng lặp code.

Một điều khác cần xem xét là nguyên tắc bạn đã học trong chương trước: **Một lớp, Một trách nhiệm**. Bằng cách đặt trách nhiệm **WinnerState** vào **SoldState**, bạn đã đưa ra trách nhiệm THỨ HAI của **SoldState**. Điều gì xảy ra khi chương trình khuyến mãi kết thúc? Hoặc kinh phí của cuộc thi thay đổi? Vì vậy, nó là một sự đánh đổi và đi đến quyết định thiết kế.



Sanity testing...

Vâng, CEO của Mighty Gumball có lẽ cần Sanity testing, nhưng đó không phải là những gì chúng ta đang nói ở đây. Hãy suy nghĩ về một số khía cạnh của **GumballMachine**:

- Chúng tôi đã có rất nhiều code trùng lặp ở các trạng thái `Sold` và `Winning` và chúng tôi có thể muốn dọn sạch chúng. Chúng ta sẽ làm điều đó bằng cách nào? Chúng ta có thể biến State thành một lớp trừu tượng và cài đặt một số hành vi mặc định cho các phương thức; dù sao, các thông báo lỗi như, "You already inserted a quarter", khách hàng sẽ không nhìn thấy. Vì vậy, tất cả các phản ứng lỗi của hành vi có thể là chung chung và được kế thừa từ lớp abstract State.
- Phương thức `dispense()` luôn được gọi, ngay cả khi xoay tay quay khi không có đồng xu. Mặc dù máy hoạt động chính xác và không phân phát kẹo trừ khi nó ở trạng thái phù hợp, chúng tôi có thể dễ dàng khắc phục điều này bằng cách cài đặt `turnCrank()` return một boolean hoặc bằng cách ném ra các exception. Bạn có nghĩ giải pháp nào là tốt hơn không?
- Tất cả các sự thông minh cho chuyển đổi trạng thái nằm bên trong các lớp State. Vấn đề này có thể gây ra điều gì? Chúng ta có muốn chuyển logic đó vào lớp Gumball Machine không? Khi thực hiện như vậy có ưu điểm và khuyết điểm gì?
- Bạn sẽ khởi tạo rất nhiều đối tượng `GumballMachine` đúng chứ? Nếu vậy, bạn có thể muốn chuyển các state instance thành các static instance và chia sẻ chúng với nhau. Những thay đổi này sẽ yêu cầu những gì từ `GumballMachine` và State?

Tối nay: Một cuộc hội ngộ Strategy và State Pattern



Strategy: Này anh bạn. Bạn có nghe tôi ở Chương 1 không?

State: Vâng, chắc chắn tôi đã nghe qua tên anh đâu đó.

Strategy: Tôi vừa mới giúp đỡ cho Template Method (https://toihocdesignpattern.com/chuong-8-template-method-pattern.html) – anh ấy cần tôi giúp để hoàn thành chương 8. Vì vậy, anh trai cao quý của tôi, anh là ai?

State: Giống như mọi khi – giúp các lớp thể hiện các hành vi khác nhau ở các trạng thái khác nhau.

Strategy: Tôi không biết điều đó, có vẻ như bạn đã làm giống những gì tôi làm và bạn chỉ sử dụng các từ khác nhau để mô tả nó. Hãy suy nghĩ điều này: Tôi cho phép các đối tượng kết hợp các hành vi hoặc thuật toán khác nhau thông qua kết hợp (composition) và ủy quyền (delegation). Bạn chỉ sao chép tôi thôi.

State: Tôi thừa nhận rằng những gì chúng ta làm chắc chắn có liên quan, nhưng ý định của tôi hoàn toàn khác với bạn. Và, cách tôi chỉ cho client của mình sử dụng kết hợp (composition) và ủy quyền (delegation) là hoàn toàn khác với anh.

Strategy: Ồ vâng? Làm thế nào? Tôi không biết điều đó.

State: Chà, nếu anh dành thêm một chút thời gian để đi ra bên ngoài, anh có thể hiểu. Dù sao, hãy suy nghĩ về cách anh làm việc: anh có một lớp được khởi tạo và anh thường cung cấp cho nó một "chiến lược" thực hiện một số hành vi. Giống như, trong Chương 1, anh đã đưa ra các hành vi tiếng vịt kêu (quack), phải không? Vịt thật thì có một tiếng kêu quack "thực sự", còn vịt cao su có một tiếng quack nhưng kêu ré lên.

Strategy: Vâng, đó là một công việc tốt… và tôi chắc chắn bạn có thể thấy làm như vậy mạnh mẽ hơn so với kế thừa hành vi của bạn, phải không?

State: Phải, tất nhiên. Bây giờ, hãy nghĩ về cách tôi làm việc; Nó khác hoàn toàn.

Strategy: Xin lỗi, bạn sẽ phải giải thích điều đó.

State: Được rồi, khi các đối tượng **Context** của tôi được tạo, tôi có thể cho chúng biết trạng thái bắt đầu, nhưng sau đó theo thời gian chúng thay đổi trạng thái của chính chúng.

Strategy: Này, thôi nào, tôi cũng có thể thay đổi hành vi trong runtime; Đó là tất cả những gì tôi đã nói về composition!

State: Chắc chắn bạn có thể, nhưng cách tôi làm việc được xây dựng xung quanh các trạng thái riêng biệt; Các đối tượng **Context** của tôi thay đổi trạng thái theo thời gian theo một số chuyển đổi trạng thái được xác định rõ. Nói cách khác, thay đổi hành vi được xây dựng theo sơ đồ của tôi – đó là cách tôi làm việc!

Strategy: Chà, tôi thừa nhận, tôi không khuyến khích các đối tượng của mình xác định rõ sự liên hệ và chuyển đổi giữa các trạng thái. Trên thực tế, tôi thường thích kiểm soát chiến lược mà các đối tượng của tôi đang sử dụng.

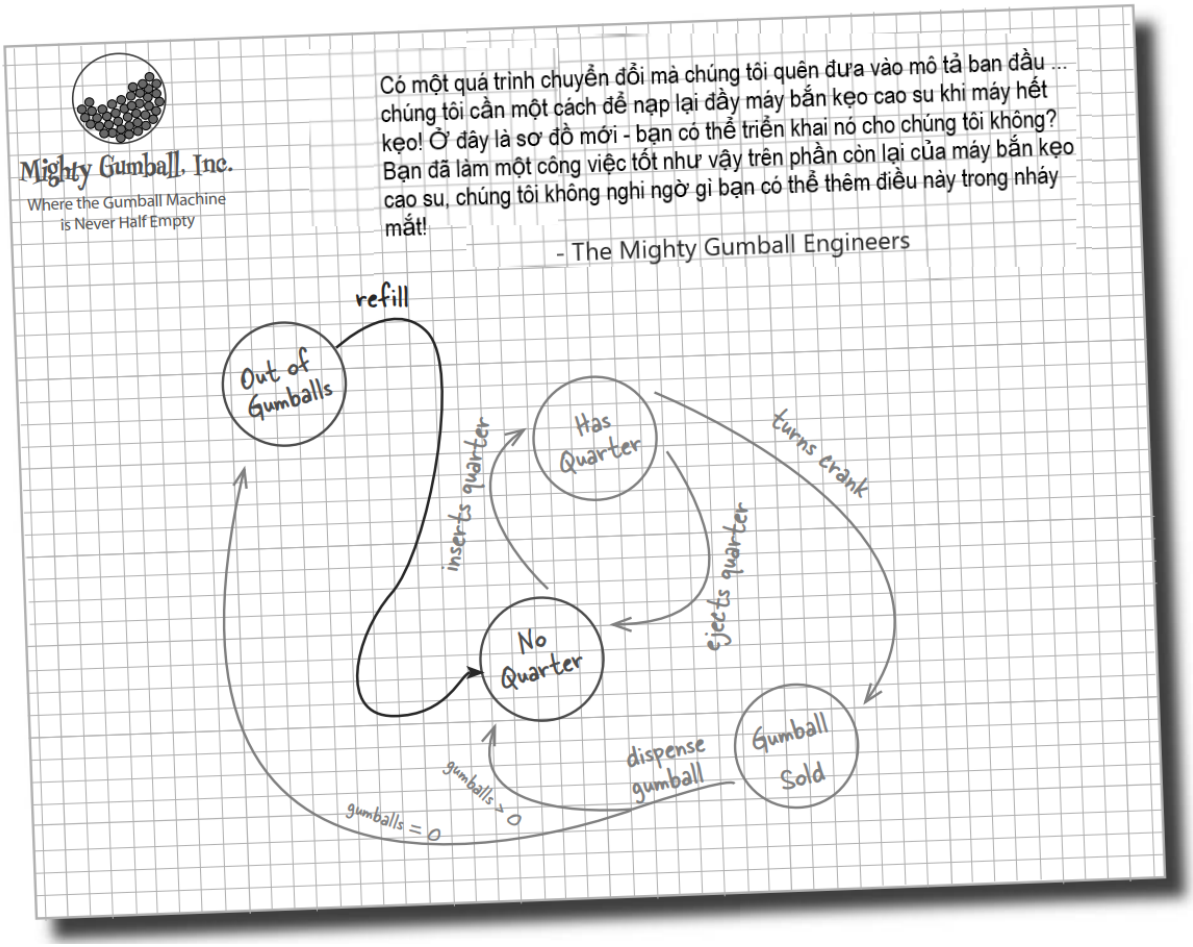
State: Hãy nhìn xem, tôi đã nói rằng chúng ta giống nhau về cấu trúc, nhưng những gì chúng ta làm hoàn toàn khác nhau về ý định. Đối mặt với nó, thế giới đã sử dụng cả hai chúng ta.

Strategy: Yeah, yeah, tiếp tục sống giấc mơ của bạn. Bạn hành động như bạn là một mẫu lớn như tôi, nhưng hãy tự nhìn lại đi: Tôi nằm trong Chương 1 (https://toihocdesignpattern.com/chuong-1-strategy-pattern-chao-mung-den-voi-design-patterns.html); họ đặt bạn trong Chương 10. Ý tôi là, có bao nhiêu người thực sự sẽ đọc đến đây?

State: Bạn đang giỡn đấy à? Đây là một cuốn sách Head First và người đọc Head First rất cừ khôi. Tất nhiên, họ sẽ đọc đến Chương 10!

Strategy: Đó mới là người anh em của tôi, luôn là người mơ mộng.

Chúng tôi gần như quên mất!



Gọt lại bút chì

Chúng tôi cần bạn viết phương thức **refill()** cho máy Gumball. Nó có một đối số (số lượng kẹo cao su mà bạn thêm vào máy) và nên cập nhật số lượng máy kẹo cao su và đặt lại trạng thái máy.

Đáp án:

```
void refill(int count) {
     this.count = count;
     state = noQuarterState;
}
```


Bạn đã làm một số công việc tuyệt vời! Tôi đã có thêm một số ý tưởng sẽ thay đổi ngành công nghiệp kẹo cao su và tôi cần bạn thực hiện chúng. Suyttttt! Tôi sẽ cho bạn biết về những ý tưởng này trong chương tiếp theo.

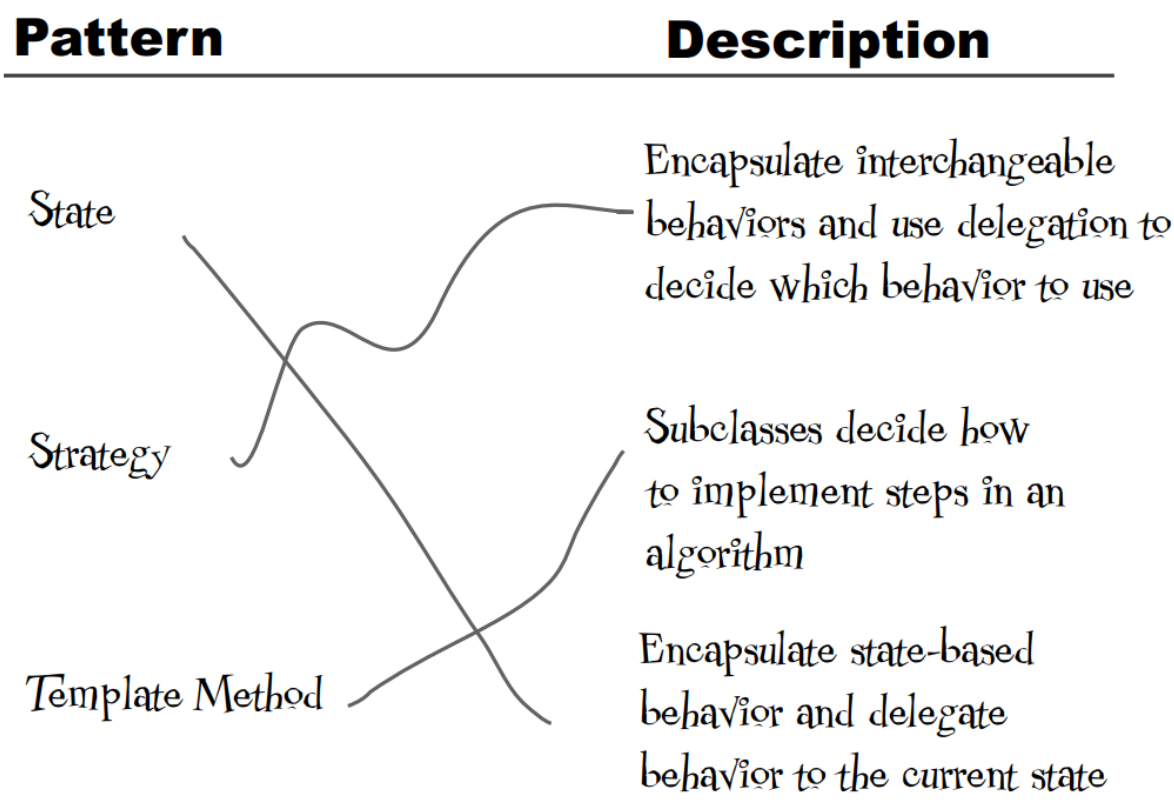
Bài tập:

WHO DOES WHAT?

Nối Mô hình với Mô tả của nó

Pattern	Description
State	Encapsulate interchangeable behaviors and use delegation to decide which behavior to use
Strategy	Subclasses decide how to implement steps in an algorithm
Template Method	Encapsulate state-based behavior and delegate behavior to the current state

Đáp án:



Tóm tắt

- State Pattern cho phép một đối tượng có nhiều hành vi khác nhau dựa trên trạng thái bên trong của nó.
- Không giống như một state machine thủ tục (if...else...), State Pattern biểu thị trạng thái dưới dạng một lớp đầy đủ.
- Context có được hành vi của nó bằng cách ủy quyền cho đối tượng trạng thái hiện tại mà nó được thiết đặt.
- Bằng cách gói gọn mỗi trạng thái vào một lớp, chúng tôi "bản địa hóa" mọi thay đổi sẽ cần phải thực hiện.
- State Pattern và Strategy Pattern (https://toihocdesignpattern.com/chuong-1-strategy-pattern-chao-mung-den-voi-design-patterns.html) có cùng sơ đồ lớp, nhưng chúng khác nhau về ý định.
- Strategy Pattern (https://toihocdesignpattern.com/chuong-1-strategy-pattern-chao-mung-den-voi-design-patterns.html) thường cài đặt các lớp Context bằng một hành vi hoặc thuật toán.
- State Pattern cho phép một Context thay đổi hành vi của nó khi trạng thái của Context thay đổi.
- Chuyển đổi trạng thái có thể được kiểm soát bởi các lớp State hoặc bởi các lớp Context.
- Sử dụng State Pattern thường sẽ dẫn đến số lượng lớp lớn hơn trong thiết kế của bạn.
- Các lớp trạng thái có thể được chia sẻ giữa các Context instance.